Esmuflily - SMuFL in LilyPond

Release v2.7

Esmuflily is an extension for LilyPond that supports SMuFL compliant fonts like Bravura or Ekmelos.
It provides switches to turn on the SMuFL support for individual types of graphical objects (clefs, note heads, flags, etc.) and it defines additional commands and styles for SMuFL glyphs. So scores can benefit from both, SMuFL's comprehensive character set and LilyPond's awesome Emmentaler font.

Accidentals and key signatures are omitted. They are supported in detail by Ekmelily.
See Cosmuflily for how to combine both.

Esmuflily requires LilyPond version 2.24 or higher.

This document uses the Ekmelos font for all SMuFL glyph.

28 October 2026

Contents

Author and License . . . 3
Usage . . . 4
Fonts . . . 5
  Font Metadata . . . 6
Commands . . . 8
  SMuFL Switches . . . 9
  Extended Text . . . 11
  Definition String . . . 12
  Orientation . . . 13
  Basic Markup Commands . . . 14
Cosmuflily . . . 20
  Cosmuflily Commands . . . 21
Musical Symbols . . . 23
System Start Delimiters . . . 24
Staff Dividers and Separators . . . 26
Colon and Segno Bar Lines . . . 27
Clefs . . . 29
  Clef Markup . . . 34
Time Signatures . . . 35
  Cadenza Signatures . . . 38
Metronome Marks . . . 39
Note Heads . . . 40
  Shape Note Heads . . . 46
  Note Name Note Heads . . . 49
  Note Clusters . . . 50
  Note Markup . . . 53
Augmentation Dots . . . 57
Flags and Grace Note Slashes . . . 58
Rests . . . 60
  Rest Markup . . . 62
Scripts and Expressive Marks . . . 63
  Expressive Mark Markup . . . 67
Dynamics . . . 68
Laissez Vibrer . . . 70
Tremolo Marks . . . 71
Stem Decorations . . . 73
Falls and Doits . . . 75
Arpeggios . . . 76
Breathing Signs and Caesuras . . . 78
Percent Repeats . . . 79
Multi-Segment Spanner . . . 80
  Trill Spans and Pitches . . . 85
Ottavation . . . 88
Tuplet Numbers . . . 92
Fingering Instructions . . . 94
String Number Indications . . . 97
Piano Pedals . . . 98
Harp Pedals . . . 100
Fret Diagrams . . . 101
Accordion Registers . . . 103
Accordion Ricochet . . . 107
Figured Bass . . . 108
Lyrics . . . 110
Chord Name Symbols . . . 111
Analytics Symbols . . . 112
Function Theory Symbols . . . 113
Percussion Symbols . . . 120
Electronic Music Symbols . . . 122
Arrows and Arrow Heads . . . 124

Author and License

Esmuflily was written by Thomas Richter, thomas-richter@aon.at
Contributor: Daniel Benjamin Miller

Copyright © 2020-2026 Thomas Richter

Esmuflily is licensed under the MIT License.
This license is copied below, and is also available in the file `LICENSE.txt` , and at mit-license.org .

> The MIT License (MIT)
>
> Copyright © 2020-2026 Thomas Richter
>
> Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the “Software”), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:
>
> The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.
>
> THE SOFTWARE IS PROVIDED “AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Usage

Esmuflily on GitHub

Install from folder `ly` the following files:

- `esmufl.ily`
- `ekmd.scm`
- `ekmd-template.scm`

`ekmd-template.scm` is required only if no metadata cache file exists for the desired font.
For some fonts, a cache file is already available:

- `ekmd-bravura.scm`
- `ekmd-ekmelos.scm`
- `ekmd-leland.scm`
- `ekmd-petaluma.scm`
- `ekmd-sebastian.scm`

Add the following lines near the top of your LilyPond input file.
`ekmFont` is optional but must be placed before `\include`.

```
ekmFont = "FONT"
\include "esmufl.ily"
```

Esmuflily + Ekmelily

See Cosmuflily to combine Esmuflily with Ekmelily for a full SMuFL support.

Fonts

Esmuflily requires a SMuFL compliant font.

It uses Ekmelos by default.
Another font can be selected, either with the variable preceding the include file

```
ekmFont = "FONT"
```

or with the command line option

```
-dekmfont=FONT
```

Both, Esmuflily and Ekmelily select a font that way. Separate fonts for graphical objects can be specified with the property `font-name`, such as

```
\override NoteHead.font-name = #"FONT"
```

> **Warning**
>
> The property `font-series` is used internally by Esmuflily for SMuFL support in several graphical objects (`NoteHead`, `Script`, `Fingering`, etc.)
> Do not override it.

Drawing paths

Esmuflily supports drawing paths instead of font glyphs, which allows to produce stand-alone SVG output. A trailing `#` in FONT switches to globally drawing paths, i.e. it effects all SMuFL output. It includes the file `FNAME-paths.ily` if available, where FNAME is the font name in all lowercase. This file must provide the Scheme procedure `ekm-path-stencil`.
Such a file can be generated with `pathtable.py` . See `ekmelos-paths.ily` for the Ekmelos font.

Paths do not support:

- Spaces and other glyphs without a contour.
- Font features like stylistic alternates and ligatures.
- Side-bearing.

Example:

```
\ekm-chars #'(#xE262 #xE566 #xEAA6 #xEAA5)
```



The path (SVG) output ignores the ligature of sharp (U+E262) and trill (U+E566), and the side-bearing of the trill span segments, so they do not overlap.

> The markup commands `\ekm-charf` and `\ekm-str` can draw paths independent of this global setting, but still require `ekm-path-stencil`.

> The markup command `\ekm-lily` cannot draw paths.

Font Metadata

Esmuflily can use font-specific metadata provided by a SMuFL compliant JSON file or by a cache file. If the latter doesn't exist, the JSON file is read and a cache file is created to be used subsequently.
In the JSON file, glyphs must be given with their canonical glyph name, not with the Unicode code point.
A cache file is a Scheme file with the metadata of interest for Esmuflily extracted from a JSON file, and additional data for font-specific musical symbols.

JSON file locations

See also SMuFL "Font-specific metadata locations" .

- MD_LOCATION/MD_NAME
- MD_LOCATION
  1. PRIVATE_LOCATION
  2. USER_LOCATION/SMuFL/Fonts/FONT
  3. SYSTEM_LOCATION/SMuFL/Fonts/FONT
- PRIVATE_LOCATION
  Can be specified either with the variable preceding the include file

  ```
  ekmMetadata = "PRIVATE_LOCATION"
  ```

  or with the command line option

  ```
  -dekmmetadata=PRIVATE_LOCATION
  ```

  A trailing `#` in PRIVATE_LOCATION forces creating a cache file even if it already exists.
- USER_LOCATION

Linux	`$XDG_DATA_HOME`
Windows	`%LOCALAPPDATA%`
macOS	`~/Library/Application Support`

- SYSTEM_LOCATION

Linux	`$XDG_DATA_DIRS`
Windows	`%CommonProgramFiles%` `%CommonProgramFiles(x86)%`
macOS	`/Library/Application Support`

- MD_NAME
  1. `FNAME_metadata.json`
  2. `FNAME.json`
  3. `metadata.json`
- FONT
  Font name in normal spelling, eg. `Bravura`
- FNAME
  Font name in all lowercase, eg. `bravura`

Cache file locations

- EKMD_LOCATION/EKMD_NAME
- EKMD_LOCATION
  1. LilyPond include directory, usually the location of `esmufl.ily`
  2. PRIVATE_LOCATION
  3. USER_LOCATION/SMuFL/Fonts/FONT
  4. SYSTEM_LOCATION/SMuFL/Fonts/FONT
- EKMD_NAME

`ekmd-FNAME.scm`	Cache file for the font FNAME
`ekmd-template.scm`	Template cache file for new fonts
`types-FNAME.scm`	Types table for the font FNAME
`types-template.scm`	Template types table for new fonts

Commands

Some commands with a corresponding LilyPond command are simpler implemented, e.g. they ignore properties, while some other commands provide additional features.

Most of the commands, in particular, all markup commands always produce SMuFL output, independent of any SMuFL switches. Other commands behave differently when the corresponding switch is turned off:

[LP] Produces normal LilyPond output.

[Err] Causes an error or produces useless output.

Some commands and properties accept a special value:

- EXTEXT: A code point, a list of code points, or markup.
- DEFINITION: A string of tokens.
- ORIENTATION: Sum of axis and direction.

All commands have the prefix `ekm` or `ekm-` .

`\ekmSmuflOn ARG`
`\ekmSmuflOff ARG`

Turn the SMuFL support on and off, respectively, for one or more types of graphical objects.
ARG is:

- a single symbol `TYPE`
- a list of symbols `'(TYPE ...)`
- a list of symbols `'(- TYPE ...)` meaning all types except for `TYPE ...`

These commands set/undo context and grob properties (usually the stencil) in the current bottom context, except for `colon` and `segno` which cannot be turned off.

Any other symbol except the following is silently ignored.

`all`	All following types
`systemstart`	System start delimiters
`colon`	Colon bar lines
`segno`	Segno bar lines
`clef`	Clefs and clef modifiers
`time`	Time signatures
`metronome`	Metronome marks
`notehead`	Note heads
`dot`	Augmentation dots
`flag`	Flags and grace note slashes
`rest`	Rests and multi-measure rests
`script`	Scripts and expressive marks
`dynamic`	Absolute dynamic marks
`lv`	Laissez vibrer
`tremolo`	Tremolo marks
`arpeggio`	Arpeggios
`percent`	Percent repeats
`textspan`	Text spans
`trill`	Trill spans and pitches
`tuplet`	Tuplet numbers
`fingering`	Fingering instructions
`stringnumber`	String number indications
`pedal`	Piano pedals
`fbass`	Figured bass
`lyric`	Lyric text
`chord`	Chord name symbols

Example

Demonstrates possible places for SMuFL switches:
a `\with` block, a `\layout` block, and in the music stream.
Note that `\ekmTremolo` works independent of the `tremolo` switch which is turned on after that.
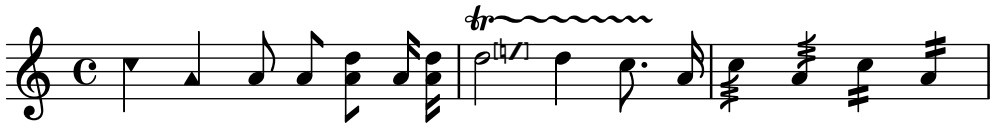
```
\new Staff \with {
  \ekmSmuflOn #'trill
}
\relative c'' {
  \ekmSmuflOn #'notehead
  \override NoteHead.style = #'triangle
  c4 a
  \ekmSmuflOff #'notehead
  \revert NoteHead.style

  \autoBeamOff
  a8
  \override Flag.style = #'straight
  a <a d> a16 <a d>
  \revert Flag.style

  \ekmPitchedTrill #'slash #'bracket
  d2 \ekmStartTrillSpan #'(-4 . 0)
  e d4 c8. a16
  \stopTrillSpan

  \ekmTremolo unmeasured
  { c4:16 a: }

  \ekmSmuflOn #'tremolo
  { c4:16 a: }
}
\layout {
  \context {
    \Score
    \ekmSmuflOn #'(flag dot)
  }
}
```



Extended Text

Some commands accept an EXTEXT value, or a pair or list of EXTEXT values.
EXTEXT can be:

- A single code point (integer).
  Calls `\ekm-char` .

  ```
  #xE046
  ```

- A list of a single code point followed by font features:
  - One or more strings.
  - A number in the range 0 to 31 of a stylistic alternate. Higher values are treated as code points (see below).
  - A negative number to draw a path instead of the font glyph. -1 draws a filled path. Any other negative number -N draws the outline with thickness N scaled to the current font size.

  Calls `\ekm-charf` .

  ```
  '(#xE242 "salt 2")

  '(#xE242 2)

  '(#xE242 -1)
  ```

- A list of one or more code points.
  Calls `\ekm-chars` .

  ```
  '(#xE262 #xE566 #xEAA6 #xEAA5)
  ```

- Any markup.

  ```
  "poco a poco"

  (markup #:box #:ekm-char #xED19)
  ```

  > The commands `\ekmTremolo` and `\ekmStem` interpret some strings as names of symbols.

- `#f`
  Draws the empty-stencil.

Definition String

Some commands and properties accept a DEFINITION value.
This is a string of one or more tokens, each consisting of one or more characters. Their corresponding symbols are stacked in a line. Any other character in the string produces a warning and only the text created so far is drawn. See also `\ekm-def`.

Some tokens are always applicable:

- Number tokens: A sequence of digits and `/` (for integer or rational number) is drawn with `\ekm-number` in a style according to the respective command (`\ekm-finger`, `\ekm-func`), or to the property `number-style`.
- Space tokens: Their default symbols emulate some Unicode whitespace.

Token	Default symbol	Unicode
<space>	`\hspace #1`	SPACE
_	`\hspace #0.17`	HAIR SPACE
__	`\hspace #0.78`	THIN SPACE
___	`\hspace #2`	EN SPACE
____	`\hspace #4`	EM SPACE
'	`#f` (nothing)	ZERO WIDTH NO-BREAK SPACE

Orientation

Some commands accept an ORIENTATION value.
This is the sum of

- Axis: X, Y, or ±0.5 for diagonal.
- Direction: ±1, or -3 for "bilateral" orientations.

The following symbols are defined for the 12 possible values. An unsupported value is substituted with `N`.

Symbol	Value	Axis	Direction
`N`	2	Y	UP
`NE`	1.5	0.5	UP
`E`	1	X	RIGHT
`SE`	0.5	-0.5	RIGHT
`S`	0	Y	DOWN
`SW`	-0.5	0.5	DOWN
`W`	-1	X	LEFT
`NW`	-1.5	-0.5	LEFT
`NS`	-2	Y	-3
`NESW`	-2.5	0.5	-3
`EW`	-3	X	-3
`SENW`	-3.5	-0.5	-3

The commands `\ekm-arrow` and `\ekm-beater` support all 12 orientations. Missing musical symbols are completed by flipping or rotating. Missing bilateral musical symbols are substituted with those for `N,NE,E,SE`.

> Currently, only the arrow style `simple` used with Ekmelos has glyphs for all 12 orientations.

Basic Markup Commands

These commands implement the underlying SMuFL output in Esmuflily.

`\ekm-char CODEPOINT`

Draw the glyph of CODEPOINT, or the point-stencil for zero.
Used properties:

- `font-size` (0)
- `font-name` (ekm:font-name)

`\ekm-charf CODEPOINT FEATURES`

Draw the glyph of CODEPOINT with font features, independent of globally drawing paths.
FEATURES is:

- a list of one or more strings.
- the number of a stylistic alternate. `#1` and `#'(1)` and `#'("salt 1")` are equivalent. `#0` and `#'()` do not set font features.
- a negative number to draw a path instead of the font glyph. `#-1` and `#'(-1)` draw a filled path. Any other negative number -N draws the outline with thickness N scaled to the current font size.

Used properties:

- `font-size` (0)
- `font-name` (ekm:font-name)

Examples:

```
\ekm-charf ##xE242 #0
```


```
\ekm-charf ##xE242 #'("salt 1")
```


```
\ekm-charf ##xE242 #'(2)
```


```
\ekm-charf ##xE242 #-20
```


`\ekm-str STRING`

Draw STRING, independent of globally drawing paths.
Used properties:

- `font-size` (0)
- `font-name` (ekm:font-name)

`\ekm-chars CODEPOINT-LIST`

Draw the glyphs of the CODEPOINTs in the list adjoined horizontally without padding, or the point-stencil for an empty list.
Used property:

- `font-size` (0)

Examples:

```
\ekm-chars #'(#xE260 #xE2B4 #xE2B2)
```


```
\ekm-chars #'(#xE262 #xE566 #xEAA6 #xEAA5)
```


```
\ekm-chars #'(#xE1F0 #xE1F7 #xE1FC #xE1F7 #xE1F4)
```


`\ekm-text EXTEXT`

Draw EXTEXT. Depending on the argument type, it calls `\ekm-char`, `\ekm-charf`, or `\ekm-chars`, or it draws markup or the empty-stencil.
Examples:

```
\ekm-text #'(#xE242 0)
```



```
\ekm-text #'(#xE242 -20)
```



```
\ekm-text #'(#xE260 #xE2B4 #xE2B2)
```



`\ekm-concat EXTEXT-LIST`

Draw the EXTEXTs in the list stacked in a line without padding.

`\ekm-line EXTEXT-LIST`

Draw the EXTEXTs in the list stacked in a line.
Used properties:

- `word-space`
- `text-direction`

Examples:

```
\ekm-line #'(#xE046 "al fine")
```

D.C. al fine

```
\ekm-line #'((#xE6D0 1) "with" #xE78E)
```



`\ekm-join-line SEPARATOR ARGS`

Draw the markups in the list ARGS with the markup SEPARATOR in between (infix) stacked in a line.
Used properties:

- `word-space`
- `text-direction`

`\ekm-combine CODEPOINT X Y CODEPOINT2`

Combine the glyphs of CODEPOINT and CODEPOINT2, where CODEPOINT2 is translated scaled by X,Y.
Examples:

```
\ekm-combine ##xECA5 #-0.5 #1.0 ##xE56E
```



```
\ekm-combine ##xEA7F #0.3 #0 ##xE87B
```



`\ekm-cchar CENTER CODEPOINT`

Draw the glyph of CODEPOINT, centered horizontally if CENTER is `CX` or `CXY`, and vertically if CENTER is `CY` or `CXY`.

`\ekm-ctext CENTER EXTEXT`

Draw EXTEXT centered like `\ekm-cchar`.
If CENTER is `STEMCX` or `STEMCXY` (intended to draw symbols on stem) then:

- a list of code points is centered only horizontally.
- a single code point (possibly with font features) is never centered.

`\ekm-number STYLE NUMBER`

Draw NUMBER (a number or string) according to STYLE (a symbol), either composed of decimal digit symbols, or a precomposed number symbol (*), or created with a markup procedure.
If there is no precomposed number symbol defined for NUMBER, it draws the digits from another style which can be overridden with property `fall-back-style`.
A non-integer number is drawn either as a common fraction or with integer and fraction part if property `mixed` is true. A plus sign is inserted if `mixed` is `'plus`.
Numerator and denominator of a fraction are aligned according to property `fraction-style`:

- `'default` or `'vertical` : Vertical with a horizontal bar and `y-padding`.
- `'none` : Vertical with `y-padding` without a bar.
- `'horizontal` or `'line` : Horizontal with a solidus and `x-padding`.
- `'diagonal` or `'slash` : Oblique with a diagonal bar and `xy-padding`, or a precomposed fraction.

Used properties:

- `font-size` (0)
- `mixed` (#t)
- `fraction-style` ('default)
- `fraction-size` (-6) relative to the font size.
- `fraction-align` (DOWN)
- `bar-thickness` (0.15)
- `x-padding` (0.2)
- `xy-padding` (0.1)
- `y-padding` (0.1)
- `word-space`
- `fall-back-style` ('())

STYLE

`'default`	4	Uses a LilyPond font, see `\ekm-lily` further below.
`'sans`	4	-"-
`'serif`	4	-"-
`'typewriter`	4	-"-
`'time`	𝍸	Time signature U+E080-U+E089
	¼	Precomposed fraction U+E097-U+E09B
`'time-turned`	𝍷	Turned time signature U+ECE0-U+ECE9
`'time-reversed`	𝍹	Reversed time signature U+ECF0-U+ECF9
`'tuplet`	𝆹	Tuplet number U+E880-U+E889
`'fingering`	4	Fingering U+ED10-U+ED15, U+ED24-U+ED27
`'fingering-italic`	4	Fingering italic U+ED80-U+ED89
`'clef-modifier`	8	Clef modifier U+E07D, U+E07E or `'fingering-italic` (*)
`'fbass`	4	Figured bass U+EA50 .. U+EA61
`'func`	4	Function theory U+EA70-U+EA79
`'string`	④	Guitar string U+E833-U+E83C, U+E84A-U+E84D or `'sans` (*)
`'scale`	4̂	Scale degree U+EF00-U+EF08 or `'sans` (*)

`\ekm-with-parens USE-STYLE STYLE ARG`

Draw markup ARG with parentheses according to USE-STYLE (a symbol) and STYLE (a symbol).
If a USE-STYLE draws symbols from another style (*), this can be overridden with property `fall-back-style`.
Used property:

- `fall-back-style` ('())

USE-STYLE

`'default`	Uses a LilyPond font, see `\ekm-lily` further below.
`'sans`	-"-
`'serif`	-"-
`'typewriter`	-"-
`'dynamic`	Dynamic mark, draws Basic Latin with padding
`'hairpin`	Dynamic hairpin U+E542-U+E545
`'clef-modifier`	Clef modifier, draws `'fingering-italic` (*)
`'fingering`	Fingering U+ED28-U+ED2B
`'fingering-italic`	Fingering italic U+ED8A-U+ED8D
`'time`	Time signature U+E092-U+E093, U+EC82-U+EC83
`'chord`	Chord symbol U+E875-U+E878
`'fbass`	Figured bass U+EA68-U+EA6B
`'func`	Function theory U+EA8F-U+EA94
`'notehead`	Note head U+E0F5-U+E0F6
`'accidental`	Accidental U+E26A-U+E26D

STYLE

`'default`	(…)
`'parens`	(…)
`'parenthesized`	(…)
`'bracket`	[…]
`'bracketed`	[…]
`'brace`	{…}
`'angle`	<…>

`\ekm-lily STYLE EXTEXT`

Draw EXTEXT with a LilyPond font according to STYLE (a symbol).

STYLE	Draw with
`'default`	`\simple`
`'sans`	`\sans`
`'serif`	`\serif` , or `\roman` in LilyPond 2.24
`'typewriter`	`\typewriter`

> This command cannot draw paths.

`\ekm-def MAP DEFINITION`

Draw a text according to DEFINITION.

MAP is an alist of EXTEXTs mapped onto tokens (strings), or a type name (a symbol). A token which is a prefix of other tokens must be arranged after them in MAP. So the correct order is "abc", "ab", "a". The tokens in MAP take precedence over number and space tokens. The special value `#f` draws nothing, i.e. the token is simply ignored.

Used property:

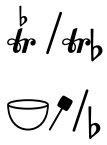
- `number-style` (#f) , see `\ekm-number`

Examples:

```
#(define my-map `(
  ("beater/" . ,(markup
                  #:ekm-beater 'timpani-hard NE))
  ("b" . #xE260)
  ("trb" . (#xE260 #xE566))
  ("tr" . #xE566)
  ("timp" . (#xE6D0 1))
  ("/" . ,(markup
            #:override '(thickness . 2)
            #:draw-line '(1 . 3)))
))

\ekm-def #my-map #"trb__/tr'b"

\ekm-def #my-map #"timpbeater//b"
```



`\ekm-label ORIENTATION LABEL ARG`

Combine markup ARG with another markup LABEL placed next to it according to ORIENTATION, where `#f` ignores the label.

Used properties:

- `font-size` (0)
- `label-size` (-4) relative to the font size.
- `padding` (0.3)

Examples:

```
\ekm-label #SE
  \ekm-char ##xE836
  "G"

\ekm-label #NW
  \ekm-beater #'timpani-hard #NE
  \ekm-char ##xE802
```



`\ekm-orient TYPE STYLE ORIENTATION`

Draw the symbol of TYPE and STYLE according to ORIENTATION.

Examples:

```
\ekm-orient #'arrow #'black #NW

\ekm-orient #'beater #'timpani-medium #NE
```



`\ekm-script STYLE DIRECTION`

Draw the musical symbol of TYPE and STYLE according to DIRECTION, where TYPE is:

- `'script` if STYLE is a string.
- `'spanner` and the edge symbol of the spanner is drawn if STYLE is a symbol.

This command is intended to combine symbols with accidentals.

Examples:

```
\ekm-script #"dmarcato" #UP
```



```
\ekm-script #'trill #UP
```


`\ekm-toe-heel-tie LEFT RIGHT`

Draw a toe-heel substitute with the EXTEXTs LEFT and RIGHT, and `x-padding`, and a tie.

The tie is either a given symbol, or it is drawn with `\tie` and the related properties (`shorten-pair`, `height-limit`, `thickness`, `offset`).

Used properties:

- `direction` (1)
- `x-padding` (0)
- `shorten-pair` ((0 . 0))
- `height-limit` (0.7)
- `thickness` (1)
- `offset` (2)

`\ekm-segment-bar SEGMENTS LENGTH AXIS`

Draw a segment bar of LENGTH (a number) according to AXIS from left to right or bottom-up.

SEGMENTS is a list `(BAR SYMBOL-START . SYMBOL-STOP)` that defines the segments. See Segment bar at Wiki for details.

Used properties:

- `font-size` (0)
- `bar-padding` (-0.08)

`\ekm-hspace WIDTH`

Draw a horizontal space of WIDTH scaled by font size.

Used property:

- `font-size` (0)

Cosmuflily

Cosmuflily = Esmuflily + Ekmelily

Combine Esmuflily with Ekmelily which supports accidentals and key signatures in several tunings, for a full SMuFL support.

Usage

Install from the Ekmelily folder `ly` the file for the desired tuning, e.g. `ekmel-24.ily` , and the main include file `ekmel-main.ily` , as well as the Esmuflily files together with `cosmufl.ily` .

Add the following lines near the top of your LilyPond input file.
`ekmFont` and `ekmUse` are optional but must be placed before `\include`.

```
ekmFont = "FONT"
ekmUse = USE
\include "cosmufl.ily"
```

USE is a sequence of tuning, notation style, and language (each is optional), followed by any number of SMuFL switches, either as a string separated by whitespace, or as a list of symbols or strings.
Tuning can also be a number. It must be the first item if any. The default is 24 (includes `ekmel-24.ily`).
Style and language are recognized only if defined in the selected tuning. The language name takes precedence in case of identical names (`arabic` in tuning "arabic" and `thm` in tuning 53). The default language is `nederlands` in most tunings. The default style is `stc` (Stein/Couper) in tuning 24.
The SMuFL switches can be preceded by a `+` , or by a `-` which turns on all but the specified switches. The default is `all` (equivalent to a sole `-`).
`cosmufl.ily` includes `esmufl.ily` and the Ekmelily file for the selected tuning, and calls `\ekmStyle`, `\language`, and `\ekmSmuflOn` (in the Score context).

USE Examples:

- Set tuning 31, Standard style, language `nederlands`, and all switches.

  ```
  ekmUse = 31
  ```

- Set tuning 24, Stein/Couper style, language `english`, and all switches.

  ```
  ekmUse = "english"
  ```

- Set tuning 5-limit JI, Extended Helmholtz-Ellis style, language `nederlands`, and all switches.

  ```
  ekmUse = "5JI he"
  ```

- Set tuning 24, Gould style, language `italiano`, and switch for note heads.

  ```
  ekmUse = #'(italiano go notehead)
  ```

- Set tuning 72, Sims style, language `svenska`, and all switches except for colon and segno bar lines.

  ```
  ekmUse = #'(72 sims svenska - colon segno)
  ```

Cosmuflily Commands

`cosmufl.ily` defines the following additional commands.

`\ekm-tuning`

Draw the selected tuning as markup.

`\ekm-combine-accidental ALTERATION-1 ALTERATION-2 ARG`

Combine markup ARG with the accidentals of ALTERATION-1 and 2 according to the selected notation style.
ALTERATION is a rational number, an alteration name (see Ekmelily), or `'none` which draws no accidental (empty-stencil).
If ALTERATION-2 is `'X` the accidental of ALTERATION-1 is placed to the right of ARG with `x-padding`. Else the accidentals are placed centered above and below ARG with `y-padding`.
Used properties:

- `x-padding` (0.2)
- `y-padding` (0.1)
- `acc-size` (-1) relative to the font size.

`\ekmScriptAccidental SCRIPT-NAME ALTERATION-1 ALTERATION-2`

Create a script from SCRIPT-NAME combined with the accidentals of ALTERATION-1 and 2.

> This does not work if SCRIPT-NAME is different from the internal name, such as `marcato` and `dmarcato`.

See also Scripts and Expressive Marks.

`\ekmStartTrillSpanAccidental TEMPO ALTERATION`

Start a trill span whose left edge symbol is combined with the accidental of ALTERATION. [LP]
See also Trill spans and pitches.

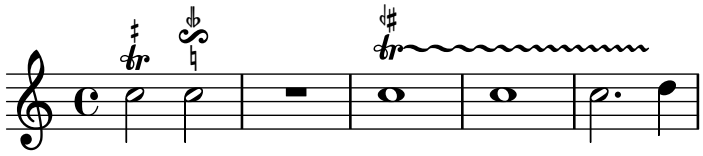
Example

Uses the default tuning 24 and language `nederlands`. Selects the Stein/Zimmermann notation style (`stz`), and turns on the SMuFL support for scripts and trill spans. The `+` is optional, since the SMuFL switches won't be recognized as language names.

```
ekmUse = #'(stz + script trill)
\include "cosmufl.ily"

\relative c'' {
  c2 ^ \ekmScriptAccidental #'trill #1/4 #'none
  c2 ^ \ekmScriptAccidental #'reverseturn #-3/4 #0
  R1

  c1 \ekmStartTrillSpanAccidental
    #'(-3 . 2)
    #'iseh
  c c2. d4
  \stopTrillSpan
}
```



Musical Symbols

A musical symbol supported by Esmuflily is defined with an EXTEXT value. This is
- a code point,
- a list of code points,
- or a markup.

All default musical symbols are SMuFL recommended characters defined with their Unicode code point.
See the Ekmelos include file `ekmelos-map.ily` with definitions to access glyphs by name.

In this document, some font-specific symbols of Ekmelos or Bravura are listed after the default symbols.

Maintenance

Esmuflily maintains the musical symbols in hierarchical categories:
- The main category is the type. Most of the types correspond to LilyPond's graphical objects, like note heads, flags, rests, and clefs.
- Sub-categories are style, duration log, direction, name (identifier), token (for definition strings), maximum size, and others.

Examples
- flag8thUp (U+E240) has type `'flag`, style `'default`, duration log `3`, and direction `UP`.
-  keyboardPedalSost (U+E659) has type `'pedal`, and token `"Sost."` .

Customize musical symbols

`\ekmMergeType TYPE TABLE`

Merge TABLE (an alist) with musical symbols for TYPE (a symbol) into the internal table, i.e. new musical symbols, styles, names, and tokens are added, and already existing ones are replaced.
See Internals at Wiki for all types and the table structure.
If TYPE is `'defaults` or `'glyphs`, TABLE is merged into the respective metadata table.
See Internals of Metadata at Wiki.
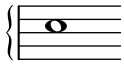
> All musical symbols are assembled in a single internal table.

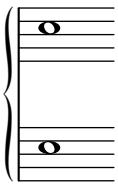
> The style `ekm` is reserved for special purpose.

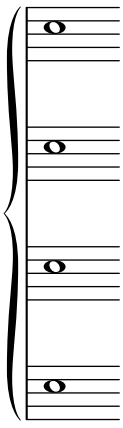
System Start Delimiters

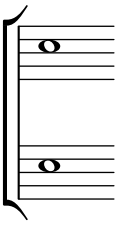
`\ekmSmuflOn #'systemstart`

Draw SMuFL system start delimiters, using `\ekm-system-start` .









`\ekm-system-start STYLE HEIGHT`

Draw a system start delimiter of HEIGHT in staff units as markup.
Used property:

- `font-size` (0)

STYLE

`'brace`		U+E000	brace
`'bracket`		U+E003	bracketTop
		U+E004	bracketBottom

with Bravura or Ekmelos

`'brace` makes use of Bravura's stylistic alternates or Ekmelos' size variants, each intended for a specific size range.

`\ekmStaffDivider DIRECTION`

Draw the next barline with an indicator to split or recombine the staff and set a `\break` .

DIRECTION

DOWN	↘	U+E00B	staffDivideArrowDown
UP	↗	U+E00C	staffDivideArrowUp
CENTER	⤢	U+E00D	staffDivideArrowUpDown

`system-separator-markup = \ekmSlashSeparator SIZE`
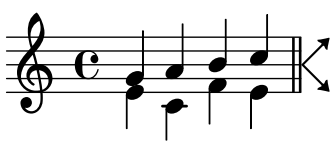
Draw a system separator mark, corresponding to SIZE (set within a `\paper` block).

SIZE

0		U+E007	systemDivider
≤ 1		U+E008	systemDividerLong
> 1		U+E009	systemDividerExtraLong

Example

```
\new Staff
<<
  \new Voice {
    \relative c'' {
      \voiceOne
      g4 a b c
      \bar "||" \ekmStaffDivider #CENTER
    }
  }
  \new Voice {
    \relative c' {
      \voiceTwo
      e4 c f e
    }
  }
>>
```



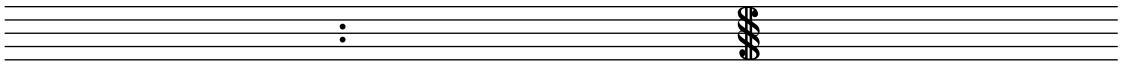
Colon and Segno Bar Lines

```
\ekmSmuflOn #'colon
\ekmSmuflOn #'segno
```

Draw SMuFL colon and segno bar lines, respectively.

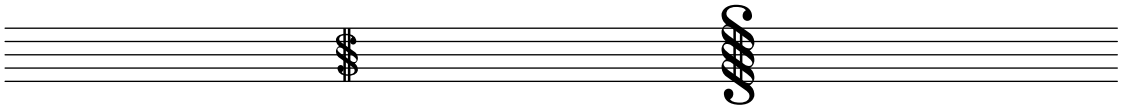
> Both switches are set independently of a context and cannot be turned off.

:	U+E043	repeatDots
S	U+E04A	segnoSerpent1



with Ekmelos

s	U+F6C8	segnoSerpentSmall1
$	U+F6CA	segnoSerpentLarge1

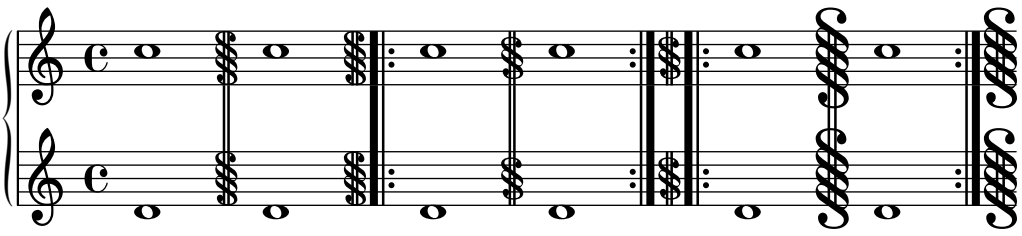


Examples

```
\new PianoStaff \with {
  \ekmSmuflOn #'segno
}
<<
  \new Staff \relative c'' {
    c1 \bar "S"
    c \bar "S.|:-S"

    % Ekmelos bar glyphs
    c \bar "s"
    c \bar ":|.s.|:-s"

    c \bar "$"
    c \bar ":|.$-$"
  }
  \new Staff \relative c' {
    d1 d d d d d
  }
>>
```
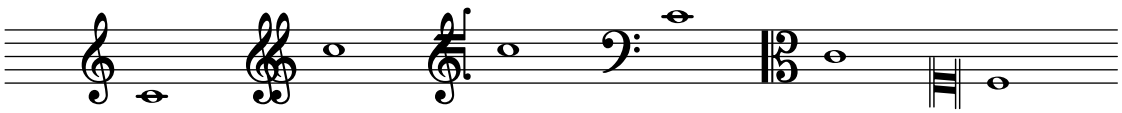


Clefs

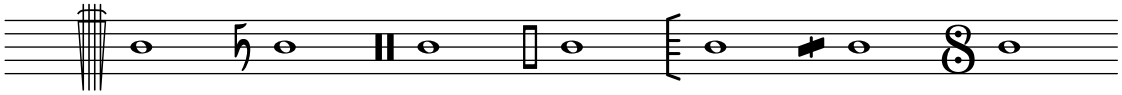
`\ekmSmuflOn #'clef`

Draw SMuFL clefs and clef modifiers.

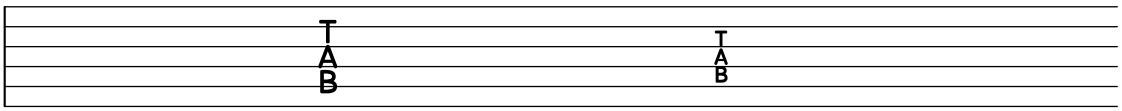
`"G" "treble" "french" …`	U+E050	gClef
`"GG"`	U+E055	gClef8vbOld
`"tenorG"`	U+E056	gClef8vbCClef
`"F" "bass" …`	U+E062	fClef
`"C" "alto" …`	U+E05C	cClef
`"neomensural-c1" …`	U+E060	cClefSquare



`"bridge"`	U+E078	bridgeClef
`"accordion"`	U+E079	accdnDiatonicClef
`"percussion"`	U+E069	unpitchedPercussionClef1
`"varpercussion"`	U+E06A	unpitchedPercussionClef2
`"semipitched"`	U+E06B	semipitchedPercussionClef1
`"varsemipitched"`	U+E06C	semipitchedPercussionClef2
`"indiandrum"`	U+ED70	indianDrumClef



`"tab"`	U+E06D	6stringTabClef
`"4stringtab"`	U+E06E	4stringTabClef



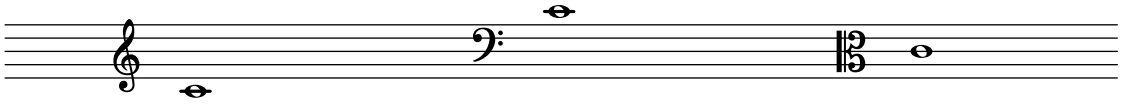
Change clefs

They are drawn either with a special glyph or with the normal glyph but smaller.
The relative font size for change clefs can be set with

```
#(set! ekm:clef-change-font-size '(SPECIAL-SIZE . NORMAL-SIZE))
```

The standard value is `'(1.5 . -2)`.

`"G"`		U+E07A	gClefChange
`"F"`		U+E07C	fClefChange
`"C"`		U+E07B	cClefChange

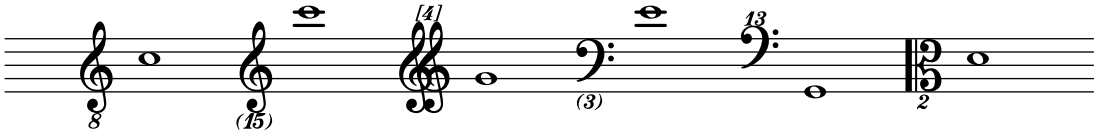


Clef modifiers

Transposition and style are drawn with the fingering italic symbols for digits, parentheses, and brackets, and with the following special symbols.

`8`	*8*	U+E07D	clef8
`15`	*15*	U+E07E	clef15

```
"G_8"
"G_(15)"
"GG^[4]"
"F_(3)"
"subbass^13"
"C_2"
```



Clef modes and attributes

`\ekmClefMode #MODE`

Set a mode for the subsequent normal, change, and cue clefs.
MODE:

- `'liga`
  Draw a clef including the modifier, either a precomposed glyph or the modifier is combined with a clef which is either a special glyph for ligating or the pure clef glyph.
- `'arrow`
  Similar to `'liga`, but the modifier is combined with the arrow variant of the G, C, or F clef.
- `'sorabji`
  Draw a Sorabji octave indicator instead of a usual modifier for the transposition 8 or 15. It is placed on the middle staff line, either next to the clef or stand-alone in a change clef (assuming that only the transposition is changing). Use the clef attribute `+` to draw the clef as well.

  > The Sorabji glyphs will be added in SMuFL 1.5 but are already define in Ekmelos.

- `'schaeffer`
  Draw the Schaeffer clef instead of a treble (G) or bass (F) clef. In a change clef, it is combined with the reversed "previous clef" (assuming that this is a change between treble and bass). Use the clef attribute `-` to omit the previous clef.
- `'default`
  Draw clef and modifier separately as usual in LilyPond.

`\ekmClefNormal`

Set the `'default` clef mode.

Clef attributes

The clef name specified with a `\clef` or `\cueClef` command can have one of the following suffixes.

- `+` or `-`
  In `'sorabji` mode, it draws / omits the clef.
  In `'schaeffer` mode, it draws / omits the "previous clef".
- `^` or `_`
  Draws the up / down arrow variant of the G, C, or F clef without a modifier.
  This is independent of the `'arrow` mode.

Examples

```
\new Staff \with {
  \ekmSmuflOn #'clef
}
\fixed c' {
  \ekmClefMode #'liga

  \clef "treble^3"
  c'1

  \cueClef "bass_6"
  c,,1

  \ekmClefMode #'arrow

  \cueClefUnset
  c'1
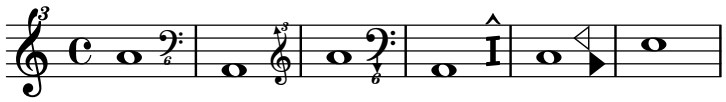
  \clef "bass_6"
  c,,1

  \ekmClefMode #'sorabji

  \clef "bass^8"
  c1

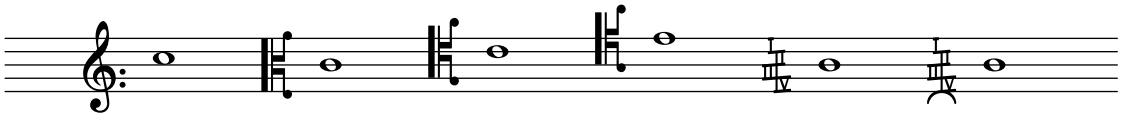
  \ekmClefMode #'schaeffer

  \clef "treble"
  c'1
}
```

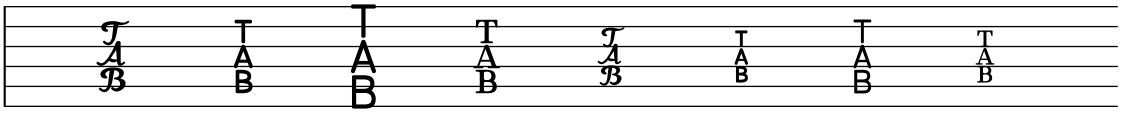


with Ekmelos
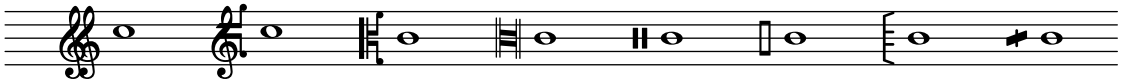
"frenchG"	U+F40E	gClef8vbFrench
"varC" "altovarC"	U+F633	cClefFrench20C
"tenorvarC"	:	
"baritonevarC"	:	
"string"	U+F71C	stringClef
"behindbridgestring"	U+F71D	behindBridgeStringClef



"tab"	U+F61E	6stringTabClefClassic
"moderntab"	U+E06D	6stringTabClef
"talltab"	U+F40A	6stringTabClefTall
"seriftab"	U+F40B	6stringTabClefSerif
"4stringtab"	U+F61F	4stringTabClefClassic
"4stringmoderntab"	U+E06E	4stringTabClef
"4stringtalltab"	U+F40C	4stringTabClefTall
"4stringseriftab"	U+F40D	4stringTabClefSerif



"GG"	U+F630	gClef8vbOldChange
"tenorG"	U+F631	gClef8vbCClefChange
"varC"	U+F634	cClefFrench20CChange
"neomensural-c3"	U+F632	cClefSquareChange
"percussion"	U+F635	unpitchedPercussionClef1Change
"varpercussion"	U+F636	unpitchedPercussionClef2Change
"semipitched"	U+F6BE	semipitchedPercussionClef1Change
"varsemipitched"	U+F6BF	semipitchedPercussionClef2Change



`\ekm-clef NAME MODE`

Draw a clef with or without a modifier according to NAME (a string) and MODE (a symbol) as markup.
NAME:

- GLYPHNAME[_change]
- GLYPHNAME_TRANSPOSITION_STYLE[_change]
- GLYPHNAME_TRANSPOSITION_STYLE_ATTRIBUTE[_change]

GLYPHNAME is the glyph name of the clef, e.g. `clefs.G` (not G).
STYLE is the parenthesis style of the modifier, see `\ekm-clef-modifier`.
ATTRIBUTE is the clef attribute.

- `on` or `off` for suffix + / − .
- `arrow` for suffix ^ / _ .

MODE is the clef mode.

- `'default` and `'schaeffer` ignore TRANSPOSITION and STYLE in NAME and draw a pure clef glyph.

`\ekm-clef-modifier TRANSPOSITION STYLE`

Draw a clef modifier with TRANSPOSITION (a number or string) and STYLE (a symbol) as markup.

STYLE

`'default`	No parentheses
`'parenthesized`	(…)
`'bracketed`	[…]

Time Signatures

`\ekmSmuflOn #'time`

Draw SMuFL time signatures.

> It supports fractional and complex time signatures, and the Scheme syntax as introduced in LilyPond 2.26. Esmuflily's own previous syntax is still available for older LilyPond.
> See `\ekmCompoundMeter` further below.

`\ekm-compound-meter TIME-SIGNATURE`

Draw a numeric time signature as markup. Sets a large plus between fractions, and a small plus within numerators.
It provides some additional property values:

- `denominator-style` : `'diagonal` or `'slash` draws fractions with a large slash.
- `nested-fraction-orientation` : `'diagonal` or `'slash` draws nested fractions with a slash or as a precomposed glyph.
- `note-head-style` : Any style of a note head or of a precomposed note.

Used properties:

- `font-size` (0)
- `denominator-style` ('default)
- `nested-fraction-mixed` (#t)
- `nested-fraction-orientation` ('default)
- `nested-fraction-relative-font-size` ('())
- `note-dots-direction` (CENTER)
- `note-flag-style` ('())
- `note-head-style` ('metronome)
- `note-staff-position` (-2)
- `bar-thickness` (0.25) used if no slash glyph is available
- `xy-padding` (0.1)

`4/4`	𝐜	U+E08A	timeSigCommon
`2/2`	𝐜	U+E08B	timeSigCutCommon
plus	+	U+E08C	timeSigPlus
	+	U+E08D	timeSigPlusSmall
slash	⁄	U+E08E	timeSigFractionalSlash
	/	U+EC84	timeSigSlash

See `\ekm-number` for time signature digits and precomposed fractions, and for the properties `bar-thickness` and `xy-padding`.

`\ekmCompoundMeter TIME-SIGNATURE`

Set a numeric time signature. A numerator can contain nested fractions for the SMuFL precomposed glyphs. If specified in a pair, e.g. `(1 . 1/2)`, this is treated as a single number without a plus sign.

> Warning
>
> This function is deprecated and should be used only with LilyPond prior to 2.26. The Scheme syntax of TIME-SIGNATURE, the style of nested fractions, and the beaming behavior are different.

The same output as with

```
\ekmCompoundMeter #'((2 4) ((1 . 1/2) 4))
```

can be achieved for LilyPond 2.26 with

```
\override Timing.TimeSignature.nested-fraction-orientation =
  #'diagonal
\time #'((2 . 4) (3/2 . 4))
\set Timing.beamExceptions = #'()
\set Timing.beatBase = #1/8
\set Timing.beatStructure = 4,3
```

Examples

```
\new Staff \with {
  \ekmSmuflOn #'time
}
\relative c'' {
  \time 5/8
  \repeat unfold 5 c8

  \time #'((1 . 4) (3 . 8))
  \repeat unfold 5 c8
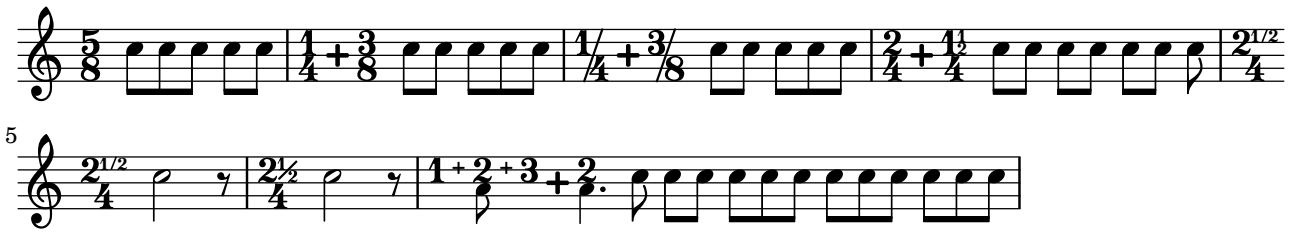
  \once \override Timing.TimeSignature.denominator-style =
    #'diagonal
  \time #'((1 . 4) (3 . 8))
  \repeat unfold 5 c8

  \time #'((2 . 4) (3/2 . 4))
  \repeat unfold 7 c8

  \override Timing.TimeSignature.nested-fraction-orientation =
    #'horizontal
  \time #'(5/2 . 4)
  c2 r8

  \override Timing.TimeSignature.nested-fraction-orientation =
    #'diagonal
  \time #'(5/2 . 4)
  c2 r8

  \override Timing.TimeSignature.denominator-style =
    #'note
  \time #'(((1 2 3) . 8) (2 . 8/3))
  \repeat unfold 12 c8
}
```



`\ekmCadenzaOn STYLE`

Start a cadenza like `\cadenzaOn` and set a SMuFL signature.

STYLE		
`"x"`	U+E09C	timeSigX
`"~"`	U+E09D	timeSigOpenPenderecki



Metronome Marks

`\ekmSmuflOn #'metronome`

Draw SMuFL notes in metronome marks specified with `\tempo`, using `\ekm-note-by-number`.

The default style of the notes is `'metronome`. Another style can be set with

```
\override Score.MetronomeMark.style = #STYLE
```

It can be any style of a note head or of a precomposed note. In the latter case, no extra stem or flag is drawn and the property `flag-style` is ignored.
The direction is always UP.

Metronome Strokes

`\ekm-metronome COUNT`

Draw COUNT metronome strokes (tickings) as markup.
Used property:

- `word-space`

with Ekmelos

	U+F614	noteTick

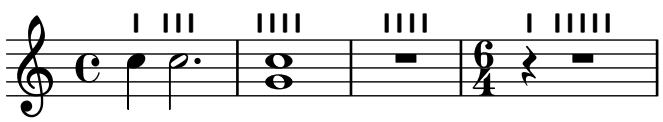
`\ekmMetronome MUSIC`

Attach metronome strokes to each note, chord, or rest in MUSIC as a horizontally centered markup above the staff, using `\ekm-metronome`. The number of strokes equals the number of quarter note values of the respective duration (possibly rounded up).

Examples

```
\relative c'' {
  \ekmMetronome {
    c4
    c2.
    <g c>1
    R1
  }

  \time 6/4
  \ekmMetronome r4
  \ekmMetronome r1*5/4
}
```



Note Heads

`\ekmSmuflOn #'notehead`

Draw SMuFL note heads. The `harmonic` and `cross` glyphs are also used with commands like `\harmonic` and `\xNote` .

The style can be set with

```
\override NoteHead.style = #STYLE
```

STYLE
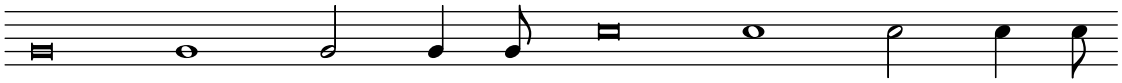
`'default`

U+E0A0 noteheadDoubleWhole
U+E0A2 noteheadWhole
U+E0A3 noteheadHalf
U+E0A4 noteheadBlack



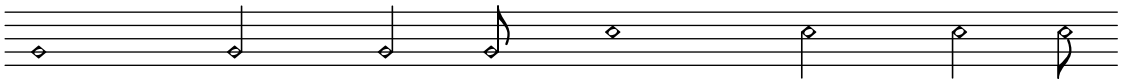
`'baroque`

U+E0A1 noteheadDoubleWholeSquare
:



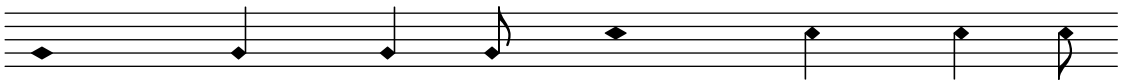
`'harmonic`

U+E0D9 noteheadDiamondHalf



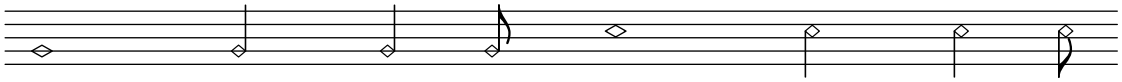
`'harmonic-black`

U+E0DC noteheadDiamondBlackWide
U+E0DB noteheadDiamondBlack
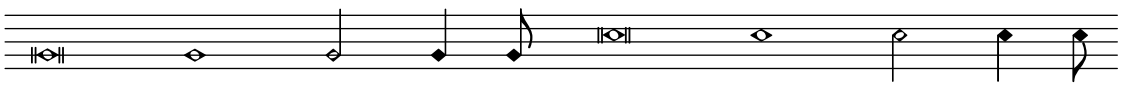


`'harmonic-white`

U+E0DE noteheadDiamondWhiteWide
U+E0DD noteheadDiamondWhite
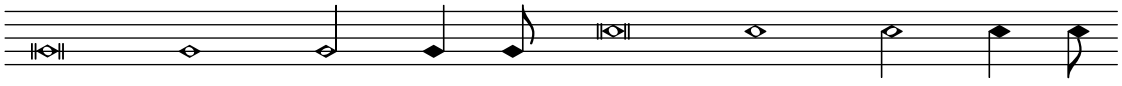


`'harmonic-mixed`

U+E0D7 noteheadDiamondDoubleWhole
U+E0D8 noteheadDiamondWhole
U+E0D9 noteheadDiamondHalf
U+E0DB noteheadDiamondBlack



`'harmonic-wide`

U+E0D7 noteheadDiamondDoubleWhole
U+E0D8 noteheadDiamondWhole
U+E0DA noteheadDiamondHalfWide
U+E0DC noteheadDiamondBlackWide
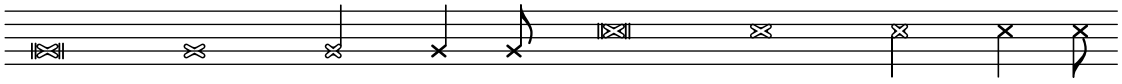


'diamond

U+E0DF	noteheadDiamondDoubleWholeOld
U+E0E0	noteheadDiamondWholeOld
U+E0E1	noteheadDiamondHalfOld
U+E0E2	noteheadDiamondBlackOld
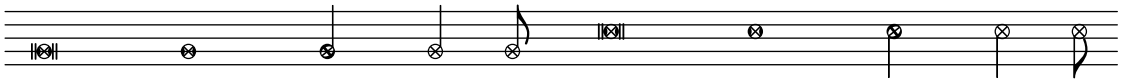


'cross

U+E0A6	noteheadXDoubleWhole
U+E0A7	noteheadXWhole
U+E0A8	noteheadXHalf
U+E0A9	noteheadXBlack



'xcircle

U+E0B0	noteheadCircleXDoubleWhole
U+E0B1	noteheadCircleXWhole
U+E0B2	noteheadCircleXHalf
U+E0B3	noteheadCircleX



'withx

U+E0B4	noteheadDoubleWholeWithX
U+E0B5	noteheadWholeWithX
U+E0B6	noteheadHalfWithX
U+E0B7	noteheadVoidWithX



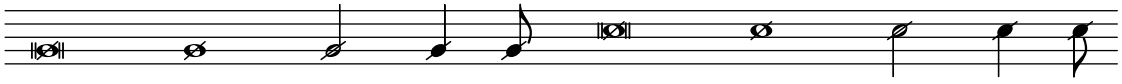
'plus

U+E0AC	noteheadPlusDoubleWhole
U+E0AD	noteheadPlusWhole
U+E0AE	noteheadPlusHalf
U+E0AF	noteheadPlusBlack
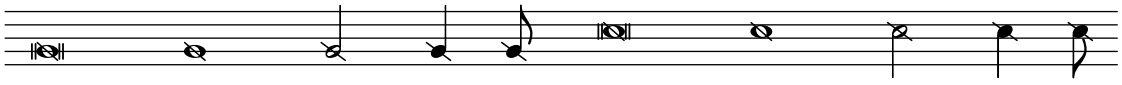


'slashed

U+E0D5	noteheadSlashedDoubleWhole1
U+E0D3	noteheadSlashedWhole1
U+E0D1	noteheadSlashedHalf1
U+E0CF	noteheadSlashedBlack1
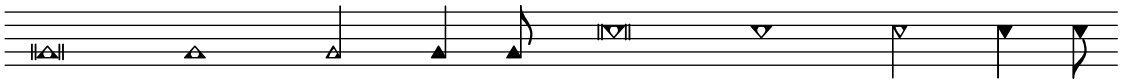


'backslashed

U+E0D6	noteheadSlashedDoubleWhole2
U+E0D4	noteheadSlashedWhole2
U+E0D2	noteheadSlashedHalf2
U+E0D0	noteheadSlashedBlack2
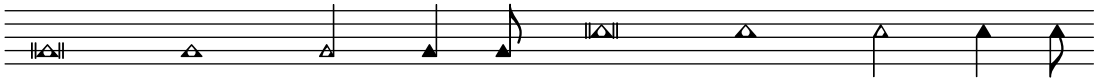


'triangle
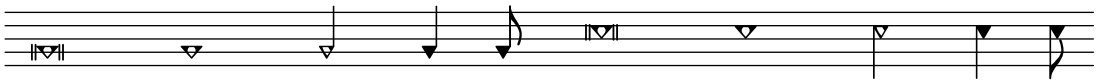
U+E0BA	noteheadTriangleUpDoubleWhole
U+E0BB	noteheadTriangleUpWhole
U+E0BC	noteheadTriangleUpHalf
U+E0BE	noteheadTriangleUpBlack
U+E0C3	noteheadTriangleDownDoubleWhole
U+E0C4	noteheadTriangleDownWhole
U+E0C5	noteheadTriangleDownHalf
U+E0C7	noteheadTriangleDownBlack



'triangle-up
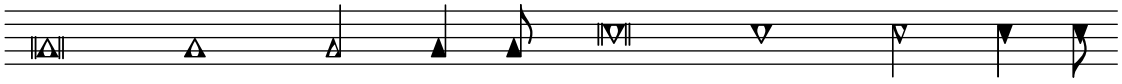


'triangle-down
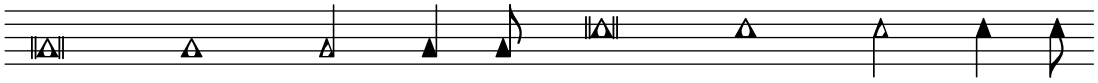


'arrow
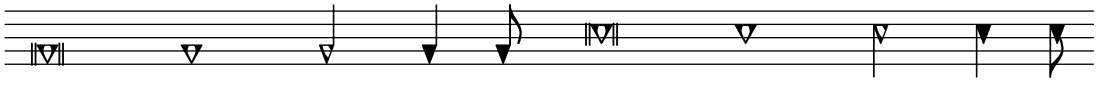
U+E0ED	noteheadLargeArrowUpDoubleWhole
U+E0EE	noteheadLargeArrowUpWhole
U+E0EF	noteheadLargeArrowUpHalf
U+E0F0	noteheadLargeArrowUpBlack
U+E0F1	noteheadLargeArrowDownDoubleWhole
U+E0F2	noteheadLargeArrowDownWhole
U+E0F3	noteheadLargeArrowDownHalf
U+E0F4	noteheadLargeArrowDownBlack



'arrow-up
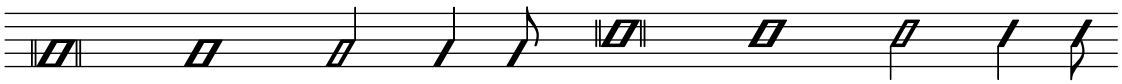


'arrow-down



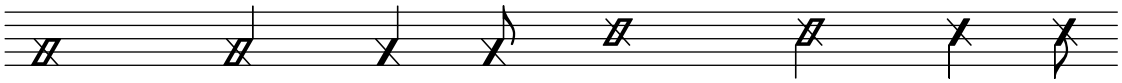
'slash

U+E10A noteheadSlashWhiteDoubleWhole
U+E102 noteheadSlashWhiteWhole
U+E103 noteheadSlashWhiteHalf
U+E101 noteheadSlashHorizontalEnds
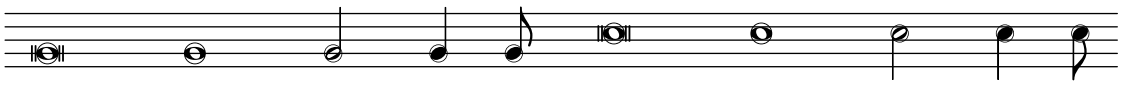


'slash-muted

U+E109 noteheadSlashWhiteMuted
U+E108 noteheadSlashHorizontalEndsMuted
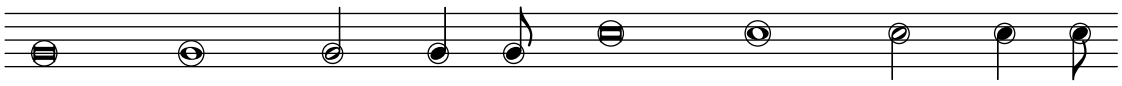


'circled

U+E0E7 noteheadCircledDoubleWhole
U+E0E6 noteheadCircledWhole
U+E0E5 noteheadCircledHalf
U+E0E4 noteheadCircledBlack



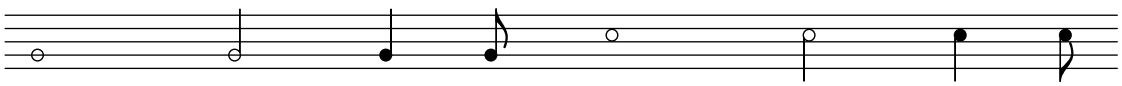
'circled-large

U+E0EB noteheadCircledDoubleWholeLarge
U+E0EA noteheadCircledWholeLarge
U+E0E9 noteheadCircledHalfLarge
U+E0E8 noteheadCircledBlackLarge



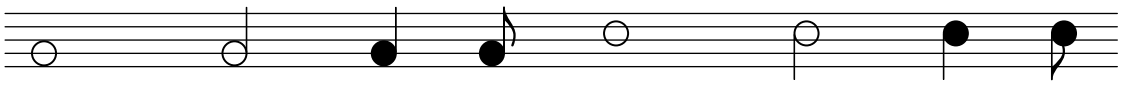
'round

U+E114 noteheadRoundWhite
U+E113 noteheadRoundBlack



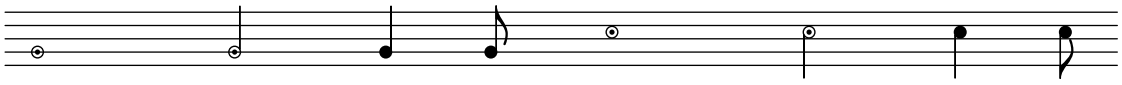
'round-large

U+E111 noteheadRoundWhiteLarge
U+E110 noteheadRoundBlackLarge



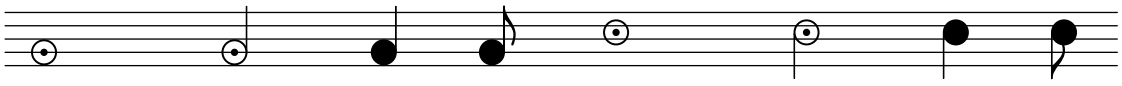
'round-dot

U+E115 noteheadRoundWhiteWithDot
U+E113 noteheadRoundBlack

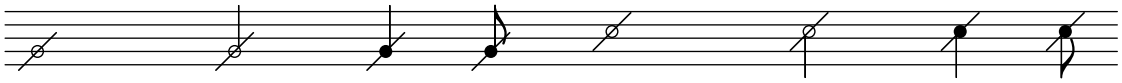


'round-dot-large
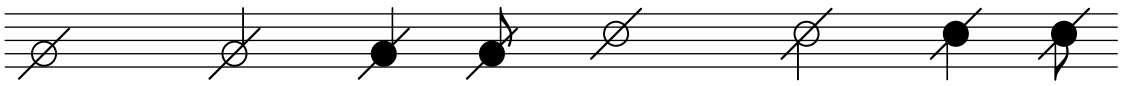
U+E112 noteheadRoundWhiteWithDotLarge
U+E110 noteheadRoundBlackLarge



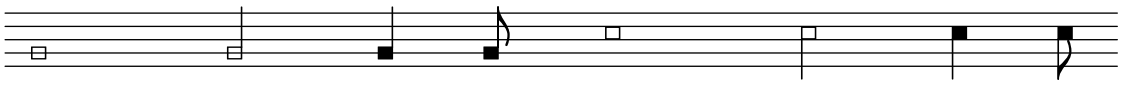
'round-slashed	U+E119	noteheadRoundWhiteSlashed
	U+E118	noteheadRoundBlackSlashed



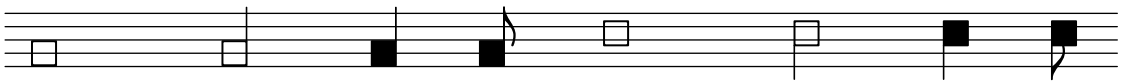
'round-slashed-large	U+E117	noteheadRoundWhiteSlashedLarge
	U+E116	noteheadRoundBlackSlashedLarge



'square	U+E0B8	noteheadSquareWhite
	U+E0B9	noteheadSquareBlack
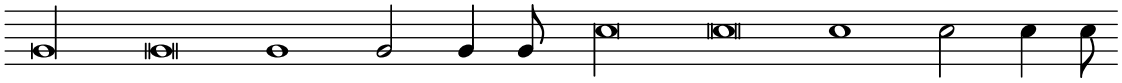


'square-large	U+E11B	noteheadSquareBlackWhite
	U+E11A	noteheadSquareBlackLarge



with Ekmelos
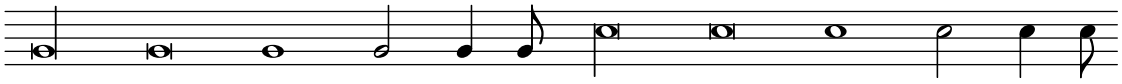
'default

U+F637 noteheadLongaUp
U+F638 noteheadLongaDown
:
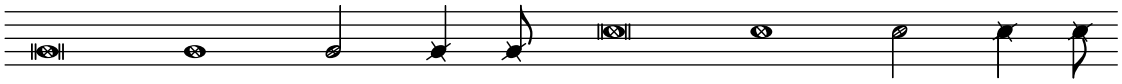


'altdefault

U+F637 noteheadLongaUp
U+F638 noteheadLongaDown
U+F639 noteheadDoubleWholeAlt
:
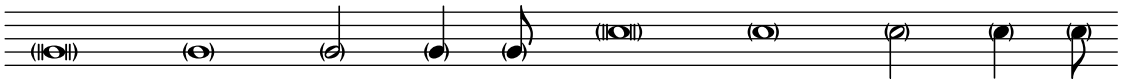


'withx-black

:
U+F680 noteheadBlackWithX



'parenthesised

U+F5DF noteheadDoubleWholeParens
U+F5DE noteheadWholeParens
U+F5DD noteheadHalfParens
U+F5DC noteheadBlackParens

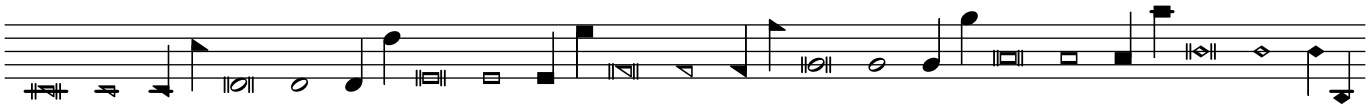


Shape Note Heads

All forms in LilyPond are supported, but some note heads of Feta don't have exact matches in SMuFL, e.g. the thin shapes of `\southernHarmonyHeads` and the reversed shapes for stem up of `\funkHeads`.

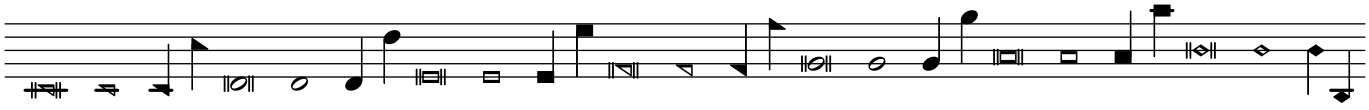
`\sacredHarpHeads`

fa	U+ECD3	noteShapeTriangleLeftDoubleWhole
	U+E1B6	noteShapeTriangleLeftWhite
	U+E1B7	noteShapeTriangleLeftBlack
	U+ECD2	noteShapeTriangleRightDoubleWhole
	U+E1B4	noteShapeTriangleRightWhite
	U+E1B5	noteShapeTriangleRightBlack
sol	U+ECD0	noteShapeRoundDoubleWhole
	U+E1B0	noteShapeRoundWhite
	U+E1B1	noteShapeRoundBlack
la	U+ECD1	noteShapeSquareDoubleWhole
	U+E1B2	noteShapeSquareWhite
	U+E1B3	noteShapeSquareBlack
mi	U+ECD4	noteShapeDiamondDoubleWhole
	U+E1B8	noteShapeDiamondWhite
	U+E1B9	noteShapeDiamondBlack



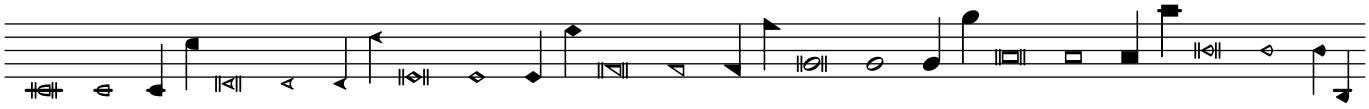
`\southernHarmonyHeads`

fa	U+ECD3	noteShapeTriangleLeftDoubleWhole
	U+E1B6	noteShapeTriangleLeftWhite
	U+E1B7	noteShapeTriangleLeftBlack
	U+ECD2	noteShapeTriangleRightDoubleWhole
	U+E1B4	noteShapeTriangleRightWhite
	U+E1B5	noteShapeTriangleRightBlack
sol	U+ECD0	noteShapeRoundDoubleWhole
	U+E1B0	noteShapeRoundWhite
	U+E1B1	noteShapeRoundBlack
la	U+ECD1	noteShapeSquareDoubleWhole
	U+E1B2	noteShapeSquareWhite
	U+E1B3	noteShapeSquareBlack
mi	U+ECD4	noteShapeDiamondDoubleWhole
	U+E1B8	noteShapeDiamondWhite
	U+E1B9	noteShapeDiamondBlack



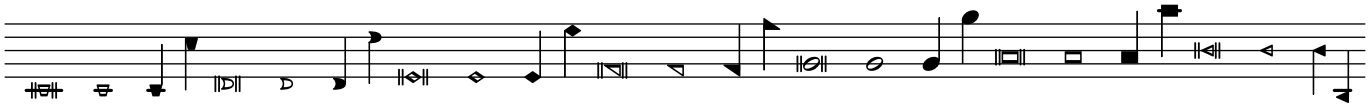
`\funkHeads`

do	U+ECDB	noteShapeMoonLeftDoubleWhole
	U+E1C6	noteShapeMoonLeftWhite
	U+E1C7	noteShapeMoonLeftBlack
re	U+ECDC	noteShapeArrowheadLeftDoubleWhole
	U+E1C8	noteShapeArrowheadLeftWhite
	U+E1C9	noteShapeArrowheadLeftBlack
mi	U+ECD4	noteShapeDiamondDoubleWhole
	U+E1B8	noteShapeDiamondWhite
	U+E1B9	noteShapeDiamondBlack
fa	U+ECD3	noteShapeTriangleLeftDoubleWhole
	U+E1B6	noteShapeTriangleLeftWhite
	U+E1B7	noteShapeTriangleLeftBlack
	U+ECD2	noteShapeTriangleRightDoubleWhole
	U+E1B4	noteShapeTriangleRightWhite
	U+E1B5	noteShapeTriangleRightBlack
sol	U+ECD0	noteShapeRoundDoubleWhole
	U+E1B0	noteShapeRoundWhite
	U+E1B1	noteShapeRoundBlack
la	U+ECD1	noteShapeSquareDoubleWhole
	U+E1B2	noteShapeSquareWhite
	U+E1B3	noteShapeSquareBlack
ti	U+ECDD	noteShapeTriangleRoundLeftDoubleWhole
	U+E1CA	noteShapeTriangleRoundLeftWhite
	U+E1CB	noteShapeTriangleRoundLeftBlack



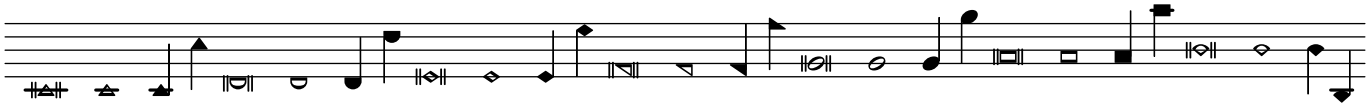
`\walkerHeads`

do	U+ECD8	noteShapeKeystoneDoubleWhole
	U+E1C0	noteShapeKeystoneWhite
	U+E1C1	noteShapeKeystoneBlack
re	U+ECD9	noteShapeQuarterMoonDoubleWhole
	U+E1C2	noteShapeQuarterMoonWhite
	U+E1C3	noteShapeQuarterMoonBlack
mi	U+ECD4	noteShapeDiamondDoubleWhole
	U+E1B8	noteShapeDiamondWhite
	U+E1B9	noteShapeDiamondBlack
fa	U+ECD3	noteShapeTriangleLeftDoubleWhole
	U+E1B6	noteShapeTriangleLeftWhite
	U+E1B7	noteShapeTriangleLeftBlack
	U+ECD2	noteShapeTriangleRightDoubleWhole
	U+E1B4	noteShapeTriangleRightWhite
	U+E1B5	noteShapeTriangleRightBlack
sol	U+ECD0	noteShapeRoundDoubleWhole
	U+E1B0	noteShapeRoundWhite
	U+E1B1	noteShapeRoundBlack
la	U+ECD1	noteShapeSquareDoubleWhole
	U+E1B2	noteShapeSquareWhite
	U+E1B3	noteShapeSquareBlack
ti	U+ECDA	noteShapeIsoscelesTriangleDoubleWhole
	U+E1C4	noteShapeIsoscelesTriangleWhite
	U+E1C5	noteShapeIsoscelesTriangleBlack



`\aikenHeads`

do	U+ECD5	noteShapeTriangleUpDoubleWhole
	U+E1BA	noteShapeTriangleUpWhite
	U+E1BB	noteShapeTriangleUpBlack
re	U+ECD6	noteShapeMoonDoubleWhole
	U+E1BC	noteShapeMoonWhite
	U+E1BD	noteShapeMoonBlack
mi	U+ECD4	noteShapeDiamondDoubleWhole
	U+E1B8	noteShapeDiamondWhite
	U+E1B9	noteShapeDiamondBlack
fa	U+ECD3	noteShapeTriangleLeftDoubleWhole
	U+E1B6	noteShapeTriangleLeftWhite
	U+E1B7	noteShapeTriangleLeftBlack
	U+ECD2	noteShapeTriangleRightDoubleWhole
	U+E1B4	noteShapeTriangleRightWhite
	U+E1B5	noteShapeTriangleRightBlack
sol	U+ECD0	noteShapeRoundDoubleWhole
	U+E1B0	noteShapeRoundWhite
	U+E1B1	noteShapeRoundBlack
la	U+ECD1	noteShapeSquareDoubleWhole
	U+E1B2	noteShapeSquareWhite
	U+E1B3	noteShapeSquareBlack
ti	U+ECD7	noteShapeTriangleRoundDoubleWhole
	U+E1BE	noteShapeTriangleRoundWhite
	U+E1BF	noteShapeTriangleRoundBlack



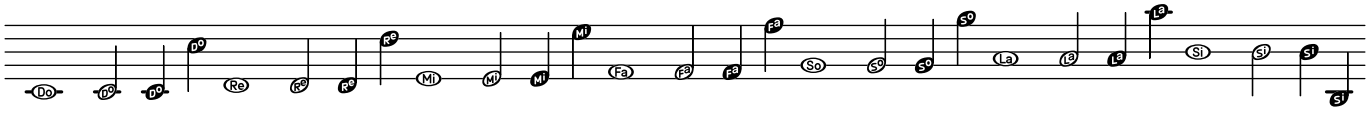
Note Name Note Heads

`\ekmNameHeads...`

Draw note heads with solfège (easy play) note names. [Err]

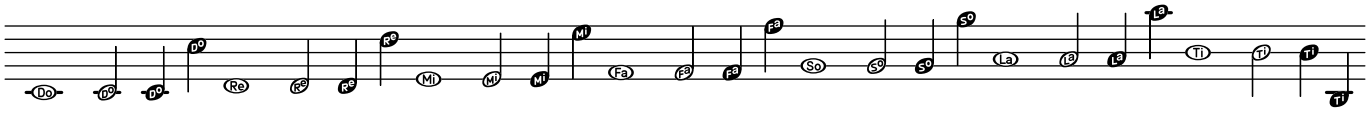
```
\ekmNameHeads
\ekmNameHeadsMinor
```

do	U+E150	noteDoWhole
	U+E158	noteDoHalf
	U+E160	noteDoBlack
re	U+E151	noteReWhole
	U+E159	noteReHalf
	U+E161	noteReBlack
mi	U+E152	noteMiWhole
	U+E15A	noteMiHalf
	U+E162	noteMiBlack
fa	U+E153	noteFaWhole
	U+E15B	noteFaHalf
	U+E163	noteFaBlack
so	U+E154	noteSoWhole
	U+E15C	noteSoHalf
	U+E164	noteSoBlack
la	U+E155	noteLaWhole
	U+E15D	noteLaHalf
	U+E165	noteLaBlack
si	U+E157	noteSiWhole
	U+E15F	noteSiHalf
	U+E167	noteSiBlack



```
\ekmNameHeadsTi
\ekmNameHeadsTiMinor
```

do ... la	:	
ti	U+E156	noteTiWhole
	U+E15E	noteTiHalf
	U+E166	noteTiBlack



Note Clusters

`\ekmCluster MUSIC`

Draw note clusters ("Cowell clusters") instead of chords in MUSIC, consisting of a bottom and a top note head. Inner notes of the chords are ignored.

`\ekmCrossCluster MUSIC`

Draw cross-staff note clusters for each note in MUSIC up to the next note above at the same moment.

The `Ekm_span_cluster_engraver` must be included in the context for this command.

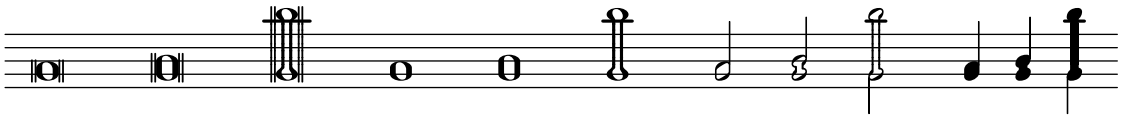
A chord with two notes in MUSIC is drawn like `\ekmCluster`.

The style can be set with

```
\override NoteHead.style = #STYLE
```

STYLE

`'default`	U+E124	noteheadClusterDoubleWhole2nd
	U+E128	noteheadClusterDoubleWhole3rd
	U+E12C	noteheadClusterDoubleWholeTop
	U+E12D	noteheadClusterDoubleWholeMiddle
	U+E12E	noteheadClusterDoubleWholeBottom
	U+E125	noteheadClusterWhole2nd
	U+E129	noteheadClusterWhole3rd
	U+E12F	noteheadClusterWholeTop
	U+E130	noteheadClusterWholeMiddle
	U+E131	noteheadClusterWholeBottom
	U+E126	noteheadClusterHalf2nd
	U+E12A	noteheadClusterHalf3rd
	U+E132	noteheadClusterHalfTop
	U+E133	noteheadClusterHalfMiddle
	U+E134	noteheadClusterHalfBottom
	U+E127	noteheadClusterQuarter2nd
	U+E12B	noteheadClusterQuarter3rd
	U+E135	noteheadClusterQuarterTop
	U+E136	noteheadClusterQuarterMiddle
	U+E137	noteheadClusterQuarterBottom

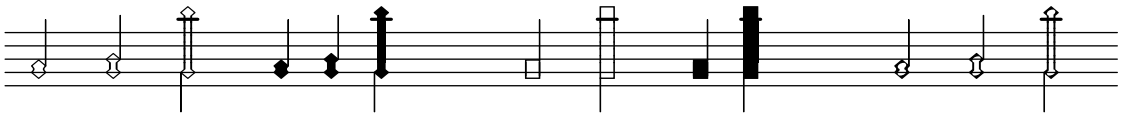


`'harmonic`	U+E138	noteheadDiamondClusterWhite2nd
	U+E13A	noteheadDiamondClusterWhite3rd
	U+E13C	noteheadDiamondClusterWhiteTop
	U+E13D	noteheadDiamondClusterWhiteMiddle
	U+E13E	noteheadDiamondClusterWhiteBottom
	U+E139	noteheadDiamondClusterBlack2nd
	U+E13B	noteheadDiamondClusterBlack3rd
	U+E13F	noteheadDiamondClusterBlackTop
	U+E140	noteheadDiamondClusterBlackMiddle
	U+E141	noteheadDiamondClusterBlackBottom
`'square`	U+E145	noteheadRectangularClusterWhiteTop
	U+E146	noteheadRectangularClusterWhiteMiddle
	U+E147	noteheadRectangularClusterWhiteBottom
	U+E142	noteheadRectangularClusterBlackTop
	U+E143	noteheadRectangularClusterBlackMiddle
	U+E144	noteheadRectangularClusterBlackBottom

with Ekmelos

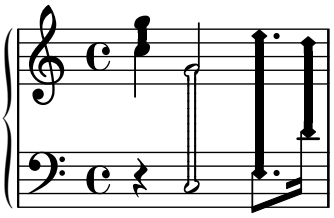
`'diamond`	U+F64B	noteheadDiamondClusterHalf2nd
	U+F64C	noteheadDiamondClusterHalf3rd
	U+F64D	noteheadDiamondClusterHalfTop
	U+F64E	noteheadDiamondClusterHalfMiddle
	U+F64F	noteheadDiamondClusterHalfBottom
	U+E139	noteheadDiamondClusterBlack2nd

:



Examples

```
\new PianoStaff
\with {
  \consists #Ekm_span_cluster_engraver
  \ekmSmuflOn #'notehead
}
<<
  \new Staff {
    \ekmCluster <c'' g''>4
    g'2
    e''8.
    d''16
  }
  \new Staff {
    \clef bass
    r4
    \ekmCrossCluster {
      c2
      \override NoteHead.style = #'harmonic
      e8.
      d'16
    }
  }
>>
```



`\ekm-note DURATION DIRECTION`

Draw a note with DURATION and stem DIRECTION as markup.

DIRECTION can be fractional for another stem length or 0 for no stem.

The property `style` can be any style of a note head or of a precomposed note (see below). In the latter case, no extra stem or flag is drawn, only the sign of DIRECTION is respected, and `flag-style` is ignored.

Used properties: See `\ekm-note-by-number`.

`\ekm-note-by-number LOG DOT-COUNT DIRECTION`

Draw a note with duration LOG, DOT-COUNT augmentation dots, and stem DIRECTION as markup.

LOG is usually in the range -1 to 10.

Used properties:

- `font-size` (0)
- `style` ('())
- `flag-style` ('())
- `dots-direction` (0)
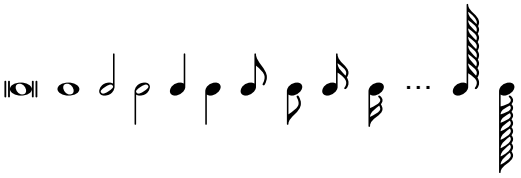
Precomposed notes

STYLE

`'note`	U+E1D0	noteDoubleWhole
	U+E1D2	noteWhole
	:	
	U+E1E6	note1024thDown



`'metronome`	U+ECA0	metNoteDoubleWhole
	U+ECA2	metNoteWhole
	:	
	U+ECB6	metNote1024thDown



with Ekmelos

`'note`



`'note-short`



`'note-straight`



`'note-beamed`



`\ekm-rhythm DEFINITION`

Draw a rhythmic pattern according to DEFINITION as markup. Space between tokens is ignored.
A pattern is composed of SMuFL beamed group notes and stem-up precomposed notes (see above), either the short stem beamed notes and `'metronome` style notes, or if DEFINITION has the prefix `*` , the long stem beamed notes and `'note` style notes.

`1`		U+ECA2	metNoteWhole
`2`		U+ECA3	metNoteHalfUp
`4`		U+ECA5	metNoteQuarterUp
`8`		U+ECA7	metNote8thUp
`16`		U+ECA9	metNote16thUp
`32`		U+ECAB	metNote32ndUp
`n`		U+E1F0	textBlackNoteShortStem
`.`		U+E1FC	textAugmentationDot
`8[`			
`8.[`			
`8..[`			
`8-`			
`16[`			
`16.[`			
`16-`			
`16+`			
`32[`			
`32-`			
`32+`			
`8]`		U+E1F2	textBlackNoteFrac8thShortStem
`16]`		U+E1F4	textBlackNoteFrac16thShortStem
`32]`		U+E1F6	textBlackNoteFrac32ndLongStem
`8_`		U+E1F7	textCont8thBeamShortStem
`16_`		U+E1F9	textCont16thBeamShortStem
`32_`		U+E1FB	textCont32ndBeamLongStem

(		U+E1FE	textTupletBracketStartShortStem
3		U+E1FF	textTuplet3ShortStem
)		U+E200	textTupletBracketEndShortStem
~		U+E1FD	textTie
=		U+E08F	timeSigEquals

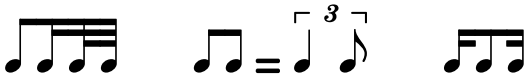
`=` draws the equal sign with space around.
`(3` draws the two glyphs with a space in between.

Examples

```
\ekm-rhythm "*8[ 16+ 32+ 32]"

\ekm-rhythm "8[ 8] = n (3 8 )"

\ekm-rhythm "16[ 8- 16]"
```



Augmentation Dots

`\ekmSmuflOn #'dot`

Draw SMuFL augmentation dots.

. . U+E1E7 augmentationDot

`\ekmSmuflOn #'flag`

Draw SMuFL flags and grace note slashes with `\slashedGrace`.
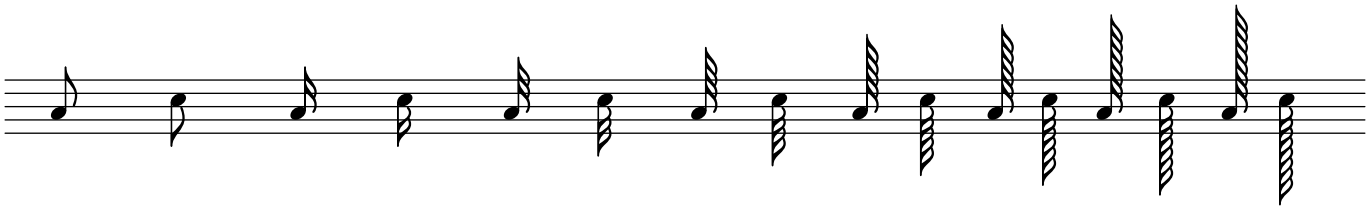
The style can be set with

```
\override Flag.style = #STYLE
```

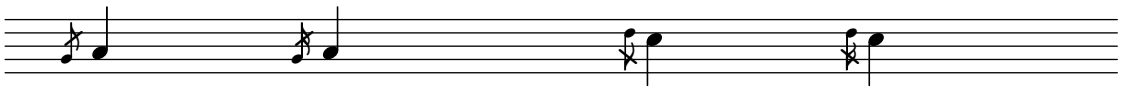
STYLE

`'default`

U+E240 flag8thUp
:
U+E24F flag1024thDown
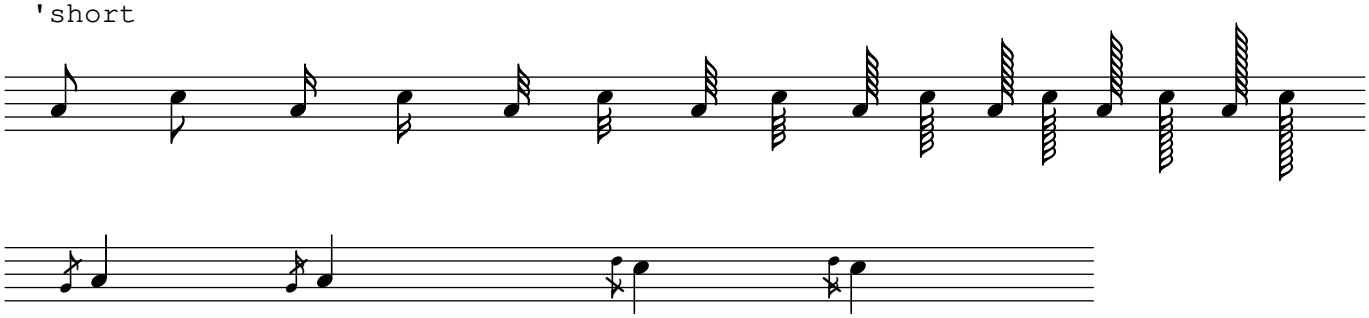


U+E564 graceNoteSlashStemUp
U+E565 graceNoteSlashStemDown
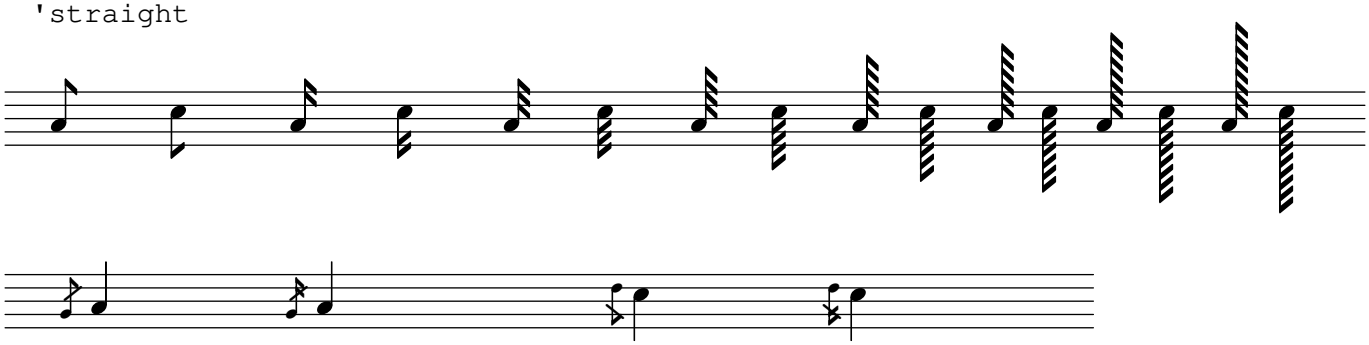


with Bravura or Ekmelos

'short



'straight



Rests

`\ekmSmuflOn #'rest`
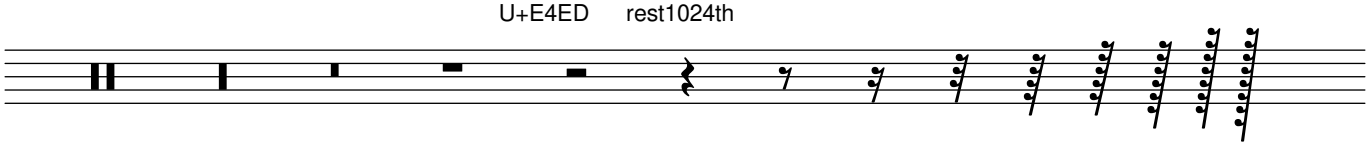
Draw SMuFL rests and multi-measure rests, as well as SMuFL time signature digits for multi-measure rest numbers.

The style can be set with

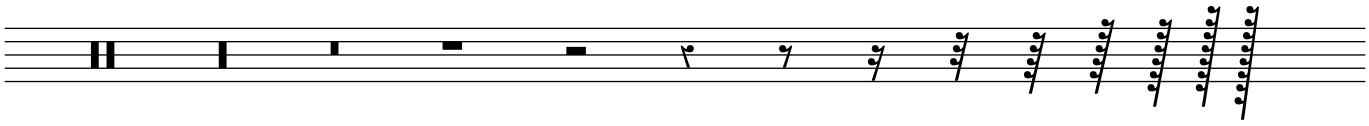
```
\override Rest.style = #STYLE
```

STYLE

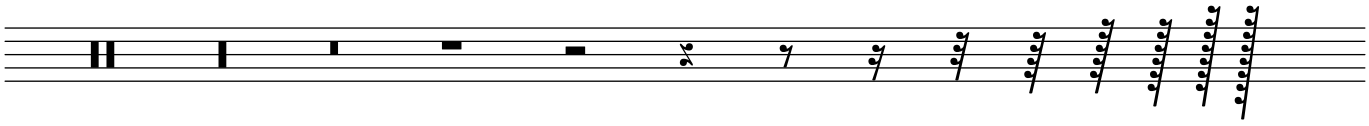
`'default` U+E4E0 restMaxima
:
U+E4ED rest1024th



`'classical` :
U+E4F2 restQuarterOld
:
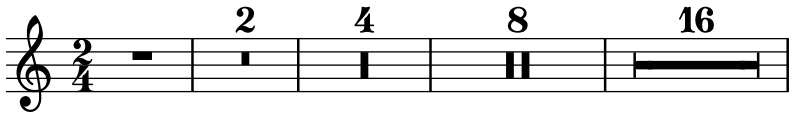


`'z` :
U+E4F6 restQuarterZ
:



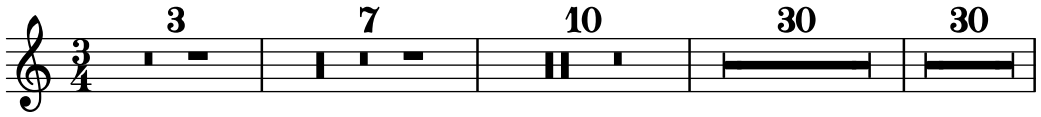
Examples

Here, the multi-measure rest numbers are SMuFL glyphs, while the time signatures are from LilyPond's Emmentaler font.

```
\relative c'' {
  \ekmSmuflOn #'rest
  \compressMMRests {
    \time 2/4
    R2 R1 R\breve R\longa R\maxima
  }
}
```




```
\relative c'' {
  \ekmSmuflOn #'rest
  \compressMMRests {
    \time 3/4
    R2.*3
    R2.*7
    R2.*10
    R2.*30
    \override MultiMeasureRest.space-increment = -2.5
    R2.*30
  }
}
```



`\ekm-rest DURATION`

Draw either a rest or a multi-measure rest with DURATION as markup.

`\ekm-rest-by-number LOG DOT-COUNT`

Draw a rest with duration LOG and DOT-COUNT augmentation dots as markup. The dots are vertically centered, contrary to `\rest-by-number` .
LOG is in the range -3 to 10.
Used properties:

- `font-size` (0)
- `ledgers` ('(-1 0 1))
- `style` ('())

`\ekm-multi-measure-rest-by-number MEASURES`

Draw a multi-measure rest as markup, with the number placed centered above unless it is 1.
Used properties:

- `font-size` (0)
- `expand-limit` (10)
- `style` ('())
- `word-space`
- `width` (8)
- `multi-measure-rest-number` (#t)

Examples

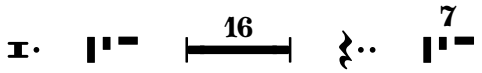
```
\ekm-rest { \breve. }

\override #'(multi-measure-rest . #t)
\override #'(multi-measure-rest-number . #f)
\ekm-rest { 1*7 }

\override #'(multi-measure-rest . #t)
\ekm-rest { 1*16 }

\ekm-rest-by-number #2 #2

\ekm-multi-measure-rest-by-number #7
```



```
\ekmSmuflOn #'script
```

Draw SMuFL scripts for expressive marks like articulations, ornamentations, performance indications, fermatas, repeat signs, etc.

```
\ekmScript SCRIPT-NAME #'(EXTEXT-UP . EXTEXT-DOWN)
\ekmScript SCRIPT-NAME EXTEXT
```

Create a script from EXTEXT, either a pair for up and down, or a single value for both directions. If the latter is a list it must be enclosed in a list. [LP]

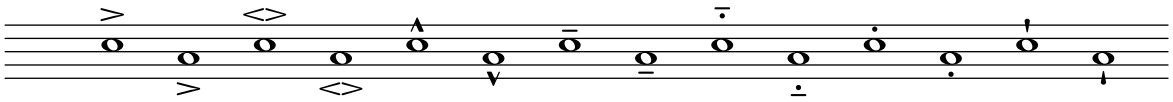
SCRIPT-NAME is the symbol of an existing script like `accent`, `marcato`, `trill`, `turn`, `upbow`, `open`, `segno`, etc. It determines the vertical positioning of the script.

```
\ekmScriptSmall SCRIPT-NAME #'(EXTEXT-UP . EXTEXT-DOWN)
\ekmScriptSmall SCRIPT-NAME EXTEXT
```

Create a script with a 3 steps smaller font size. [LP]
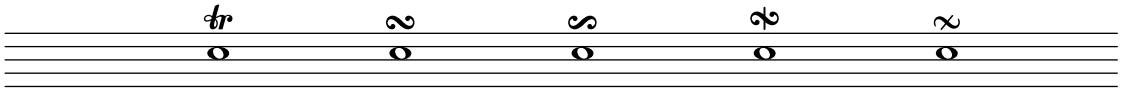
Articulations

`\accent`	U+E4A0	articAccentAbove
	U+E4A1	articAccentBelow
`\espressivo`	U+ED40	articSoftAccentAbove
	U+ED41	articSoftAccentBelow
`\marcato`	U+E4AC	articMarcatoAbove
	U+E4AD	articMarcatoBelow
`\tenuto`	U+E4A4	articTenutoAbove
	U+E4A5	articTenutoBelow
`\portato`	U+E4B2	articTenutoStaccatoAbove
	U+E4B3	articTenutoStaccatoBelow
`\staccato`	U+E4A2	articStaccatoAbove
	U+E4A3	articStaccatoBelow
`\staccatissimo`	U+E4A6	articStaccatissimoAbove
	U+E4A7	articStaccatissimoBelow



Ornamentations

`\trill`	U+E566	ornamentTrill
`\turn`	U+E567	ornamentTurn
`\reverseturn`	U+E568	ornamentTurnInverted
`\slashturn`	U+E569	ornamentTurnSlash
`\haydnturn`	U+E56F	ornamentHaydn

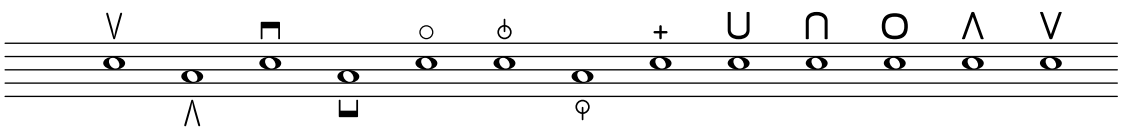


\prall		U+E56C	ornamentShortTrill
\prallprall		U+E56E	ornamentTremblement
\mordent		U+E56D	ornamentMordent
\prallmordent		U+E5BD	ornamentPrecompTrillWithMordent
\upprall		U+E59A	ornamentBottomLeftConcaveStroke
	2 ×	U+E59D	ornamentZigZagLineNoRightEnd
		U+E59E	ornamentZigZagLineWithRightEnd
\downprall		U+E5C6	ornamentPrecompMordentUpperPrefix
\upmordent		U+E5B8	ornamentPrecompSlideTrillBach
\downmordent		U+E5C7	ornamentPrecompInvertedMordentUpperPrefix
\prallup	3 ×	U+E59D	ornamentZigZagLineNoRightEnd
		U+E5A4	ornamentRightVerticalStroke
\pralldown		U+E5C8	ornamentPrecompTrillLowerSuffix
\lineprall		U+E5B2	ornamentPrecompAppoggTrill



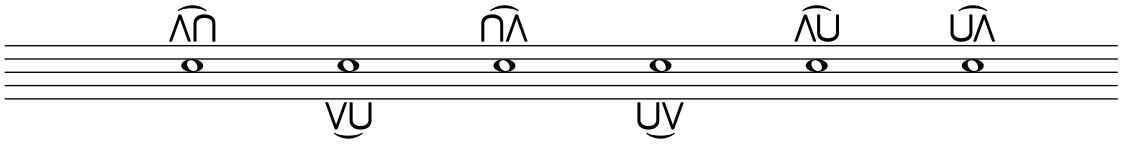
Performance indications

\upbow	U+E612	stringsUpBow
	U+E613	stringsUpBowTurned
\downbow	U+E610	stringsDownBow
	U+E611	stringsDownBowTurned
\flageolet	U+E614	stringsHarmonic
\snappizzicato	U+E631	pluckedSnapPizzicatoAbove
	U+E630	pluckedSnapPizzicatoBelow
\stopped	U+E633	pluckedLeftHandPizzicato
\heel \lheel	U+E661	keyboardPedalHeel1
\varheel \rheel	U+E662	keyboardPedalHeel2
\heelcircle	U+E663	keyboardPedalHeel3
\toe \rtoe	U+E665	keyboardPedalToe2
\vartoe \ltoe	U+E664	keyboardPedalToe1



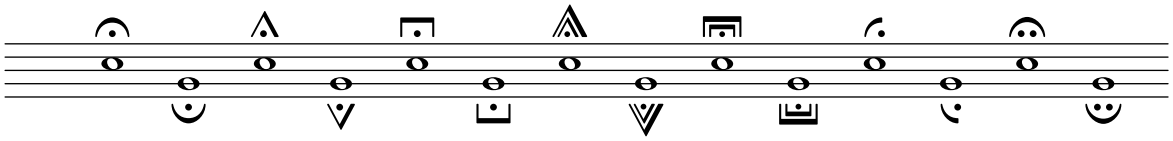
Toe-Heel substitutes

`\rtoeheel`	U+E665	keyboardPedalToe2
	U+E662	keyboardPedalHeel2
`\ltoeheel`	U+E664	keyboardPedalToe1
	U+E661	keyboardPedalHeel1
`\rheeltoe`	U+E662	keyboardPedalHeel2
	U+E665	keyboardPedalToe2
`\lheeltoe`	U+E661	keyboardPedalHeel1
	U+E664	keyboardPedalToe1
`\tweak toe-heel-style #'standard \rtoeheel`	U+E675	keyboardPedalToeToHeel
`\tweak toe-heel-style #'standard \rheeltoe`	U+E674	keyboardPedalHeelToToe



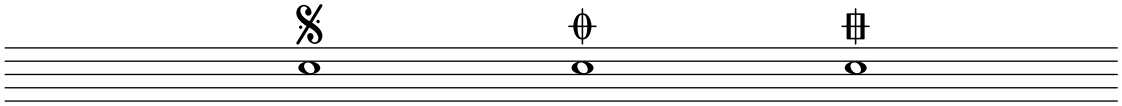
Fermatas

`\fermata`	U+E4C0	fermataAbove
	U+E4C1	fermataBelow
`\shortfermata`	U+E4C4	fermataShortAbove
	U+E4C5	fermataShortBelow
`\longfermata`	U+E4C6	fermataLongAbove
	U+E4C7	fermataLongBelow
`\veryshortfermata`	U+E4C2	fermataVeryShortAbove
	U+E4C3	fermataVeryShortBelow
`\verylongfermata`	U+E4C8	fermataVeryLongAbove
	U+E4C9	fermataVeryLongBelow
`\henzeshortfermata`	U+E4CC	fermataShortHenzeAbove
	U+E4CD	fermataShortHenzeBelow
`\henzelongfermata`	U+E4CA	fermataLongHenzeAbove
	U+E4CB	fermataLongHenzeBelow



Repeat signs

`\segno`	U+E047	segno
`\coda`	U+E048	coda
`\varcoda`	U+E049	codaSquare



Examples

```
marcatoTenuto =
  \ekmScript #'marcato #'(#xE4BC . #xE4BD)

stringsChangeBowDirection =
  \ekmScript #'downbow #'((#xE626 1))

\relative c'' {
  \ekmSmuflOn #'script
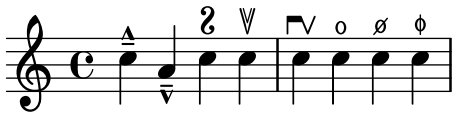
  c4 ^ \marcatoTenuto
  a4 _ \marcatoTenuto

  c4 ^ \ekmScript #'turn ##xE56A

  c4 ^ \ekmScript #'upbow ##xE61C

  c4 ^ \stringsChangeBowDirection

  % use Ekmelos glyphs
  c4 ^ \open
  c4 ^ \halfopen
  c4 ^ #(make-articulation 'halfopenvertical)
}
```



`\ekm-fermata STYLE`

Draw a fermata as markup.

Used property:

- `direction`

STYLE

`'default`		U+E4C0	fermataAbove
		U+E4C1	fermataBelow
`'short`		U+E4C4	fermataShortAbove
		U+E4C5	fermataShortBelow
`'long`		U+E4C6	fermataLongAbove
		U+E4C7	fermataLongBelow
`'veryshort`		U+E4C2	fermataVeryShortAbove
		U+E4C3	fermataVeryShortBelow
`'verylong`		U+E4C8	fermataVeryLongAbove
		U+E4C9	fermataVeryLongBelow
`'henzeshort`		U+E4CC	fermataShortHenzeAbove
		U+E4CD	fermataShortHenzeBelow
`'henzelong`		U+E4CA	fermataLongHenzeAbove
		U+E4CB	fermataLongHenzeBelow

with Ekmelos

`'extrashort`		U+F69E	fermataExtraShortAbove
		U+F69F	fermataExtraShortBelow
`'extralong`		U+F6A0	fermataExtraLongAbove
		U+F6A1	fermataExtraLongBelow

`\ekm-eyeglasses DIRECTION`

Draw eyeglasses as markup.

DIRECTION

any value		U+EC62	miscEyeglasses

with Ekmelos

`RIGHT`		U+F65F	miscEyeglassesRight

`\ekmSmuflOn #'dynamic`

Draw SMuFL absolute dynamic marks, using `\ekm-dynamic` .

`\ekm-dynamic DEFINITION`

Draw a dynamic symbol according to DEFINITION as markup.
DEFINITION must be either a single token or a sequence of the letters f, m, n, p, r, s, z, whose corresponding symbols are concatenated. This is slightly different from the usual interpretation of definition strings.

`p`		U+E520	dynamicPiano
`m`		U+E521	dynamicMezzo
`f`		U+E522	dynamicForte
`r`		U+E523	dynamicRinforzando
`s`		U+E524	dynamicSforzando
`z`		U+E525	dynamicZ
`n`		U+E526	dynamicNiente
`mp`		U+E52C	dynamicMP
`mf`		U+E52D	dynamicMF
`pf`		U+E52E	dynamicPF
`fp`		U+E534	dynamicFortePiano
`pppppp`		U+E527	dynamicPPPPPP
	:		
`pp`		U+E52B	dynamicPP
`ff`		U+E52F	dynamicFF
	:		
`ffffff`		U+E533	dynamicFFFFFF
`fz`		U+E535	dynamicForzando
`sf`		U+E536	dynamicSforzando1
`sfp`		U+E537	dynamicSforzandoPiano
`sfpp`		U+E538	dynamicSforzandoPianissimo
`sfz`		U+E539	dynamicSforzato
`sfzp`		U+E53A	dynamicSforzatoPiano
`sffz`		U+E53B	dynamicSforzatoFF
`rf`		U+E53C	dynamicRinforzando1
`rfz`		U+E53D	dynamicRinforzando2

with Ekmelos

`sff`		U+F645	dynamicSforzandoFF
`sp`		U+F646	dynamicSP
`spp`		U+F647	dynamicSPP
`sfffz`		U+F6F4	dynamicSforzatoFFF
`sffffz`		U+F6F5	dynamicSforzatoFFFF

`\ekmParensDyn STYLE DYNAMIC-MARK`

Draw DYNAMIC-MARK (an event) parenthesized with the normal text font.

STYLE

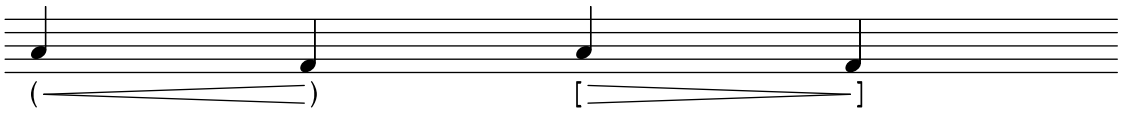
`'default`	***( )***
`'bracket`	***[ ]***
`'brace`	***{ }***
`'angle`	***< >***

`\ekmParensHairpin STYLE`

Draw the subsequent hairpin parenthesized.

STYLE

`'default`	U+E542	dynamicHairpinParenthesisLeft
	U+E543	dynamicHairpinParenthesisRight
`'bracket`	U+E544	dynamicHairpinBracketLeft
	U+E545	dynamicHairpinBracketRight



Laissez Vibrer

`\ekmSmuflOn #'lv`

Draw SMuFL laissez vibrer ties.

`\ekmLaissezVibrer SIZE`

Draw a laissez vibrer tie after a note, corresponding to SIZE. [LP]

`\laissezVibrer` is equivalent to `\ekmLaissezVibrer #0`

SIZE

any number	⁀	U+E4BA	articLaissezVibrerAbove
	‿	U+E4BB	articLaissezVibrerBelow

with Ekmelos
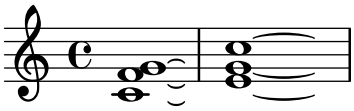
≤ 5.5	⁀	U+E4BA	articLaissezVibrerAbove
	‿	U+E4BB	articLaissezVibrerBelow
≤ 8	⁀	U+F6FC	articLaissezVibrerAboveLong
	‿	U+F6FD	articLaissezVibrerBelowLong
> 8	⁀	U+F6FE	articLaissezVibrerAboveExtraLong
	‿	U+F6FF	articLaissezVibrerBelowExtraLong

Examples

```
\relative c' {
  \ekmSmuflOn #'lv

  <c f g>1 \ekmLaissezVibrer #0

  % use Ekmelos glyphs
  <e g c>1 \ekmLaissezVibrer #9
}
```



Tremolo Marks

`\ekmSmuflOn #'tremolo`

Draw SMuFL tremolo marks on stems, except for beamed notes.
For cue notes, the tremolo marks are taken from style `'cue` if provided by the selected font. Else they are composed using the symbol with 1 flag of the current style in a compressed shape.

The style can be set with

```
\override StemTremolo.shape = #STYLE
```

STYLE

`'beam-like`

:8		U+E220	tremolo1
	:		
:128		U+E224	tremolo5

`'fingered`

:8		U+E225	tremoloFingered1
	:		
:128		U+E229	tremoloFingered5
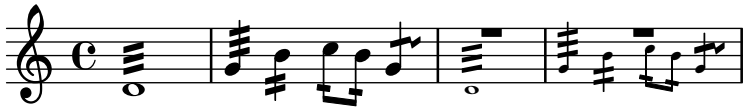
`\ekmTremolo EXTEXT MUSIC`

Draw a tremolo mark from EXTEXT on the stems of the tremolo notes in MUSIC, independent of the subdivision `:N`, and independent of the `tremolo` switch. A list of code points or a markup is centered horizontally, while a single code point is assumed being a centered stem decoration.
Some strings are defined as names of symbols:

`buzzroll`		U+E22A	buzzRoll
`penderecki`		U+E22B	pendereckiTremolo
`stockhausen`		U+E232	stockhausenTremolo
`unmeasured`		U+E22C	unmeasuredTremolo
`unmeasuredS`		U+E22D	unmeasuredTremoloSimple

Examples

```
\new Staff \with {
  \ekmSmuflOn #'tremolo
}
\relative c' {
  d1:32
  g4:32 b:16
  c8:16 b:16
  \ekmTremolo stockhausen g4:16
  <<
    {
      \override MultiMeasureRest.staff-position = #2
      R1*2
    }
    \new CueVoice \relative c' {
      d1:32
      g4:32 b:16
      c8:16 b:16
      \ekmTremolo stockhausen g4:16
    }
  >>
}
```



Stem Decorations

`\ekmStem EXTEXT MUSIC`

Draw a symbol from EXTEXT vertically centered on the stems in MUSIC. A list of code points or a markup is centered horizontally, while a single code point is assumed being a centered stem decoration. Some strings are defined as names of symbols:

`sprechgesang`		U+E645	vocalSprechgesang
`halbGesungen`		U+E64B	vocalHalbGesungen
`sussurando`		U+E646	vocalsSussurando
`damp`		U+E63B	pluckedDampOnStem
`stringNoise`		U+E694	harpStringNoiseStem
`multiphonics`		U+E607	windMultiphonicsBlackStem
`bowBehindBridge`		U+E618	stringsBowBehindBridge
`bowOnBridge`		U+E619	stringsBowOnBridge
`bowOnTailpiece`		U+E61A	stringsBowOnTailpiece
`fouette`		U+E622	stringsFouette
`vibrato`		U+E623	stringsVibratoPulse
`rimShot`		U+E7FD	pictRimShotOnStem
`crush`		U+E80C	pictCrushStem
`deadNote`		U+E80D	pictDeadNoteStem
`swish`		U+E808	pictSwishStem
`turnRight`		U+E809	pictTurnRightStem
`turnLeft`		U+E80A	pictTurnLeftStem
`turnRightLeft`		U+E80B	pictTurnRightLeftStem

Examples

```
\relative c'' {
  \ekmTremolo ##xE233
  { b4:8 a: }
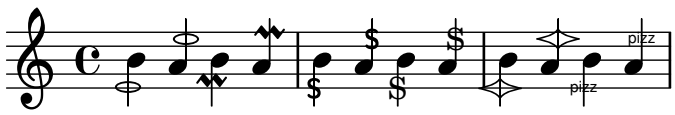
  % uses a list to center horizontally
  \ekmTremolo #'(#xE56C)
  { b4:8 a: }

  \ekmStem sussurando
  { b4 a }

  \ekmStem "S"
  { b a }

  \ekmStem #'(#xE843 #xE844)
  { b a }

  \ekmStem \markup { \fontsize #-6 \sans "pizz" }
  { b a }
}
```



Falls and Doits

`\ekmBendAfter STYLE DIRECTION`

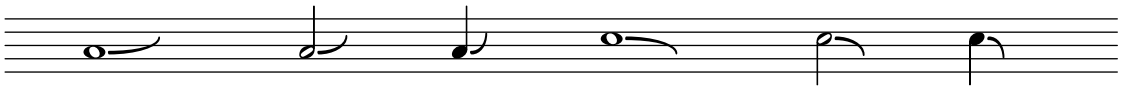
Draw a fall or doit (lift) symbol after a note.

Only the sign of DIRECTION is respected, contrary to `\bendAfter`.

STYLE

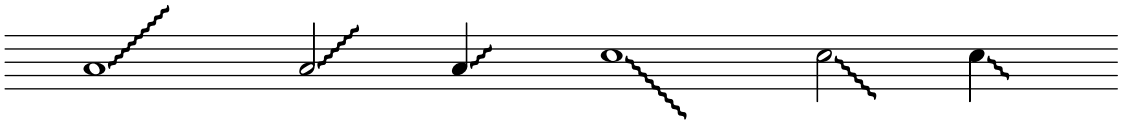
`'bend`

UP	U+E5D6	brassDoitLong
	U+E5D5	brassDoitMedium
	U+E5D4	brassDoitShort
DOWN	U+E5D9	brassFallLipLong
	U+E5D8	brassFallLipMedium
	U+E5D7	brassFallLipShort



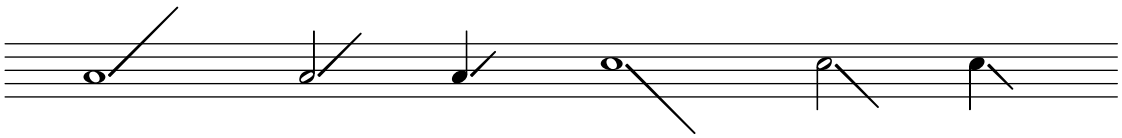
`'rough`

UP	U+E5D3	brassLiftLong
	U+E5D2	brassLiftMedium
	U+E5D1	brassLiftShort
DOWN	U+E5DF	brassFallRoughLong
	U+E5DE	brassFallRoughMedium
	U+E5DD	brassFallRoughShort



`'smooth`

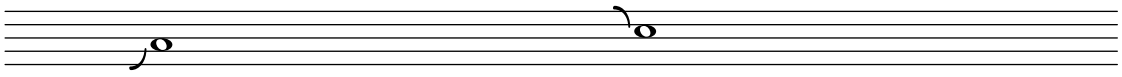
UP	U+E5EE	brassLiftSmoothLong
	U+E5ED	brassLiftSmoothMedium
	U+E5EC	brassLiftSmoothShort
DOWN	U+E5DC	brassFallSmoothLong
	U+E5DB	brassFallSmoothMedium
	U+E5DA	brassFallSmoothShort



`\ekmScoop DIRECTION MUSIC`

Draw a scoop or plop symbol to the left of each note in MUSIC.

UP	U+E5D0	brassScoop
DOWN	U+E5E0	brassPlop



Arpeggios

`\ekmSmuflOn #'arpeggio`

Draw SMuFL arpeggios.

The style can be set with

```
\override Arpeggio.style = #STYLE
```

STYLE

`'default`		U+EAA9	wiggleArpeggiatoUp
		U+EAAA	wiggleArpeggiatoDown
		U+EAAD	wiggleArpeggiatoUpArrow
		U+EAAE	wiggleArpeggiatoDownArrow
`'swash`	:		
		U+EAAB	wiggleArpeggiatoUpSwash
		U+EAAC	wiggleArpeggiatoDownSwash

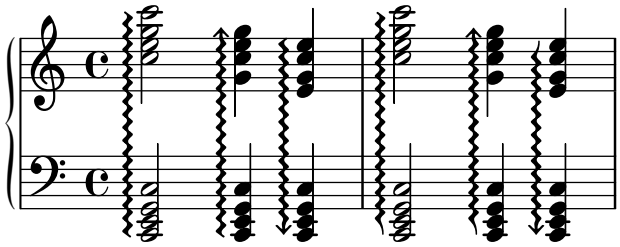
Examples

```
\new PianoStaff \with {
  \ekmSmuflOn #'arpeggio
}
<<
  \set PianoStaff.connectArpeggios = ##t

  \new Staff \relative c'' {
    <c e g c>2 \arpeggio
    \once \override PianoStaff.Arpeggio.arpeggio-direction = #UP
    <g c e g>4 \arpeggio
    \once \override PianoStaff.Arpeggio.arpeggio-direction = #DOWN
    <e g c e>4 \arpeggio

    \override PianoStaff.Arpeggio.style = #'swash
    <c' e g c>2 \arpeggio
    \once \override PianoStaff.Arpeggio.arpeggio-direction = #UP
    <g c e g>4 \arpeggio
    \once \override PianoStaff.Arpeggio.arpeggio-direction = #DOWN
    <e g c e>4 \arpeggio
  }
  \new Staff \relative c, {
    \clef bass
    <c e g c>2 \arpeggio
    <c e g c>4 \arpeggio
    <c e g c>4 \arpeggio

    <c e g c>2 \arpeggio
    <c e g c>4 \arpeggio
    <c e g c>4 \arpeggio
  }
>>
```
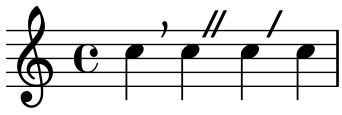


`\ekmBreathing EXTEXT`

Draw a breathing sign or caesura from EXTEXT .

Examples

```
\relative c'' {
  c4
  \ekmBreathing ##xE4CE
  c
  \ekmBreathing ##xE4D1
  c
  \ekmBreathing #'(#xE4D1 1)
  c
}
```



Percent Repeats

`\ekmSmuflOn #'percent`

Draw SMuFL percent repeats.



U+E504	repeatBarSlash
U+E500	repeat1Bar
U+E501	repeat2Bars

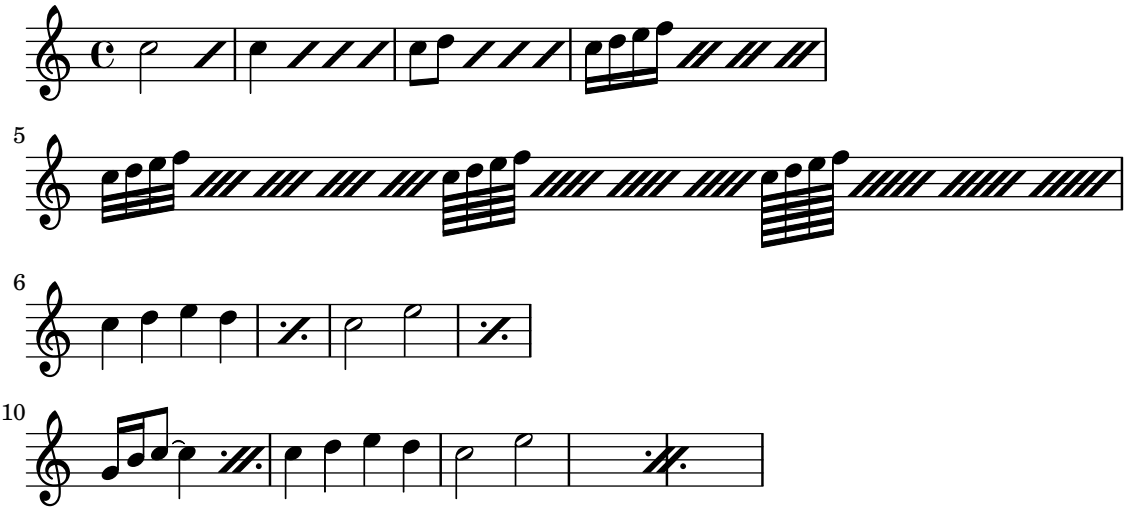
Examples

```
\relative c'' {
  \ekmSmuflOn #'percent

  \repeat percent 2 { c2 }
  \repeat percent 4 { c4 }
  \repeat percent 4 { c8 d }
  \repeat percent 4 { c16 d e f }
  \repeat percent 5 { c32 d e f }
  \repeat percent 4 { c64 d e f }
  \repeat percent 4 { c128 d e f }
  \break

  \repeat percent 2 { c4 d e d }
  \repeat percent 2 { c2 e }
  \break

  \repeat percent 2 { g,16 b c8~ c4 }
  \repeat percent 2 { c4 d e d | c2 e }
}
```



`\ekmSmuflOn #'textspan`

Draw text spans assembled from SMuFL multi-segment glyphs.

`\ekmStartSpan STYLE TEMPO ATTACHMENT`

Start a text span or trill span. [LP]
STYLE `'trill` starts a trill span. A second style can be added to draw other glyphs, e.g. `'trill-vibrato-large` . Any other style starts a text span. An undefined style like `'dashed-line` produces normal LilyPond output.
TEMPO is a number or a pair of numbers (rounded to integer) for the segments of the spanner. 0 is the main (medium) segment. Positive values mean faster (narrower) segments. Negative values mean slower (wider) segments. A pair `'(A . B)` draws all segments from A through B, evenly distributed over the spanner. The available range of numbers depends on the style and the selected font.
ATTACHMENT is an EXTEXT or a pair `'(EXTEXT-LEFT . EXTEXT-RIGHT)` for the edge symbols. A single EXTEXT is equivalent to `'(EXTEXT . 0)`. It must be specified in a pair if itself is a list.
`#f` draws the standard glyph on the left and right edge according to the style.

`\ekmStartSpanMusic STYLE TEMPO ATTACHMENT MUSIC`

Start a text span or trill span at MUSIC.
This is a music function that doesn't need the `textspan` or `trill` SMuFL switch turned on.

STYLE

`'trill`

left	𝆗𝅥	U+E566	ornamentTrill
4	𝆖	U+EAA0	wiggleTrillFastest
	:		
0	𝆖	U+EAA4	wiggleTrill
	:		
−4	𝆖	U+EAA8	wiggleTrillSlowest

`'vibrato`

left	𝆮	U+EACC	wiggleVibratoStart
3	𝆯	U+EADB	wiggleVibratoMediumFastest
	:		
0	𝆯	U+EADE	wiggleVibratoMediumFast
	:		
−3	𝆯	U+EAE1	wiggleVibratoMediumSlowest

`'vibrato-small`

left	𝆮	U+EACC	wiggleVibratoStart
3	𝆯	U+EAD4	wiggleVibratoSmallFastest
	:		
0	𝆯	U+EAD7	wiggleVibratoSmallFast
	:		
−3	𝆯	U+EADA	wiggleVibratoSmallSlowest

'vibrato-smallest

left		U+EACC	wiggleVibratoStart
3		U+EACD	wiggleVibratoSmallestFastest
	:		
0		U+EAD0	wiggleVibratoSmallestFast
	:		
−3		U+EAD3	wiggleVibratoSmallestSlowest

'vibrato-large

left		U+EACC	wiggleVibratoStart
3		U+EAE2	wiggleVibratoLargeFastest
	:		
0		U+EAE5	wiggleVibratoLargeFast
	:		
−3		U+EAE8	wiggleVibratoLargeSlowest

'vibrato-largest

left		U+EACC	wiggleVibratoStart
3		U+EAE9	wiggleVibratoLargestFastest
	:		
0		U+EAEC	wiggleVibratoLargestFast
	:		
−3		U+EAEF	wiggleVibratoLargestSlowest

'circular

left		U+EAC4	wiggleCircularStart
4		U+EACA	wiggleCircularSmall
	:		
0		U+EAC9	wiggleCircular
	:		
−4		U+EAC5	wiggleCircularLargest
right		U+EACB	wiggleCircularEnd

'circular-constant

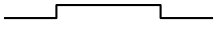
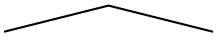
2		U+EAC2	wiggleCircularConstantLarge
1		U+EAC0	wiggleCircularConstant
0		U+EAC0	wiggleCircularConstant
−1		U+EAC1	wiggleCircularConstantFlipped
−2		U+EAC3	wiggleCircularConstantFlippedLarge

'wavy

1		U+EAB4	wiggleWavyNarrow
0		U+EAB5	wiggleWavy
−1		U+EAB6	wiggleWavyWide

'square			
1		U+EAB7	wiggleSquareWaveNarrow
0		U+EAB8	wiggleSquareWave
−1		U+EAB9	wiggleSquareWaveWide
'sawtooth			
1		U+EABA	wiggleSawtoothNarrow
0		U+EABB	wiggleSawtooth
−1		U+EABC	wiggleSawtoothWide
'beam			
7		U+EB02	beamAccelRit15
	:		
0		U+EAFB	beamAccelRit8
	:		
−7		U+EAF4	beamAccelRit1
right		U+EB03	beamAccelRitFinal

with Ekmelos

`'wavy`			
`6 .. 4`		U+F7A1	wiggleWavyExtraNarrow
`3`		U+F7A0	wiggleWavyVeryNarrow
`2`		U+F6B3	wiggleWavyNarrower
`1`		U+EAB4	wiggleWavyNarrow
`0`		U+EAB5	wiggleWavy
`-1`		U+EAB6	wiggleWavyWide
`-2`		U+F6B4	wiggleWavyWider
	:		
`-6`		U+F727	wiggleWavyQuadrupleWide
`'square`			
`6 .. 4`		U+F7A3	wiggleSquareWaveExtraNarrow
`3`		U+F7A2	wiggleSquareWaveVeryNarrow
`2`		U+F6B5	wiggleSquareWaveNarrower
`1`		U+EAB7	wiggleSquareWaveNarrow
`0`		U+EAB8	wiggleSquareWave
`-1`		U+EAB9	wiggleSquareWaveWide
`-2`		U+F6B6	wiggleSquareWaveWider
	:		
`-6`		U+F72B	wiggleSquareWaveQuadrupleWide
`'sawtooth`			
`6 .. 4`		U+F7A5	wiggleSawtoothExtraNarrow
`3`		U+F7A4	wiggleSawtoothVeryNarrow
`2`		U+F6B7	wiggleSawtoothNarrower
`1`		U+EABA	wiggleSawtoothNarrow
`0`		U+EABB	wiggleSawtooth
`-1`		U+EABC	wiggleSawtoothWide
`-2`		U+F6B8	wiggleSawtoothWider
	:		
`-6`		U+F72F	wiggleSawtoothQuadrupleWide

Examples

```
\relative c'' {
  \ekmSmuflOn #'textspan

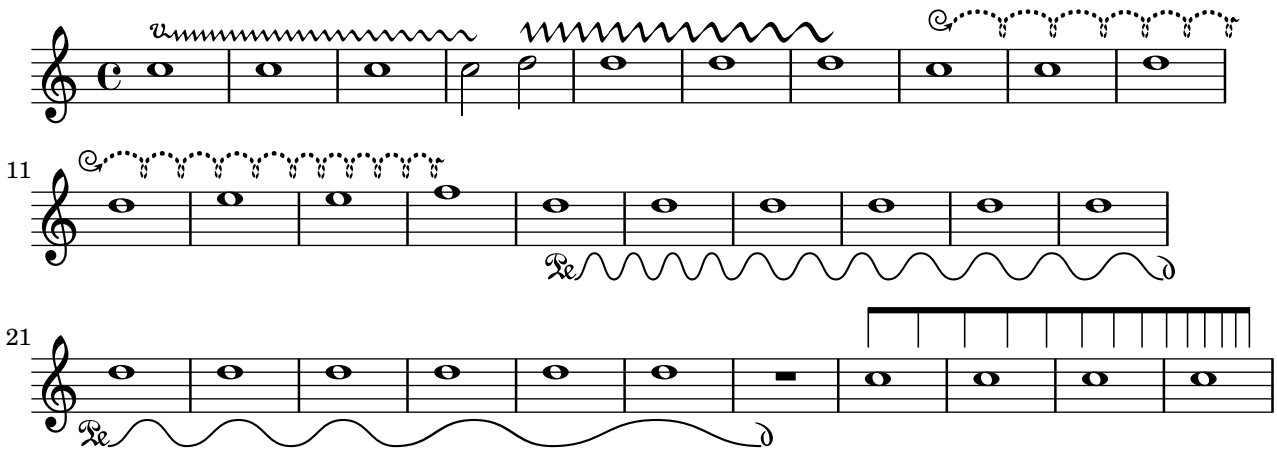
  c1 \ekmStartSpan
       #'vibrato
       #'(3 . -3)
       ##f
  c c c2 \stopTextSpan

  d2 \ekmStartSpan
       #'vibrato-large
       #'(3 . -3)
       #0
  d1 d d \stopTextSpan

  c1 \ekmStartSpan
       #'circular
       #'(-4 . -1)
       ##f
  c d \break d e e f \stopTextSpan

  \textSpannerDown
  d1 \ekmStartSpan
       #'wavy
       #'(0 . -6)  % this range uses Ekmelos glyphs
       #'((#xE651 #xE652) . #xE653)
  d d d d d \break d d d d d d
  R1 \stopTextSpan
  \textSpannerNeutral

  c1 \ekmStartSpan
       #'beam
       #'(-5 . 5)
       ##f
  c c c \stopTextSpan
}
```



Trill Spans and Pitches

`\ekmSmuflOn #'trill`

Draw trill spans assembled from SMuFL multi-segment glyphs, and SMuFL trill pitches.

`\ekmStartTrillSpan TEMPO`

Start a trill span with the style `'trill`. See also Multi-segment spanner. [LP]
TEMPO is a number or a pair of numbers (rounded to integer) for the segments of the spanner, usually in the range 4 to -4.

`\startTrillSpan` is equivalent to `\ekmStartTrillSpan #0`
and to `\ekmStartSpan #'trill #0 ##f`

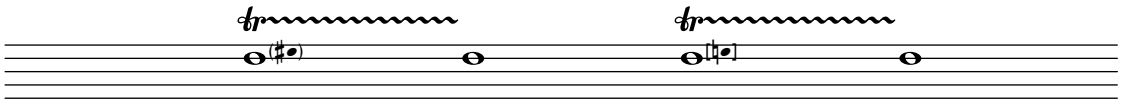
`\ekmPitchedTrill NOTEHEAD-STYLE PARENS-STYLE MAIN-NOTE AUXILIARY-NOTE`

Draw a trill pitch. [Err]
NOTEHEAD-STYLE: See Note Heads.
For variable accidentals on AUXILIARY-NOTE see Cosmuflily.

`\pitchedTrill` is equivalent to `\ekmPitchedTrill #'default #'default`

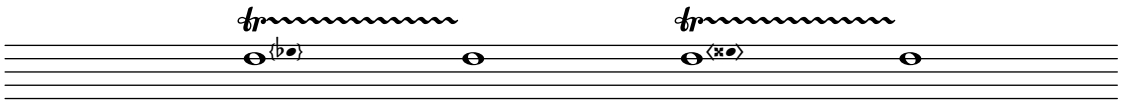
PARENS-STYLE

`'default`	U+E26A	accidentalParensLeft
	U+E26B	accidentalParensRight
`'bracket`	U+E26C	accidentalBracketLeft
	U+E26D	accidentalBracketRight



with Ekmelos

`'brace`	U+F6D4	accidentalBraceLeft
	U+F6D5	accidentalBraceRight
`'angle`	U+F6D6	accidentalAngleLeft
	U+F6D7	accidentalAngleRight



Examples

```
\relative c'' {
  \ekmSmuflOn #'trill

  c1 \ekmStartTrillSpan #0
  c c2. d4 \stopTrillSpan
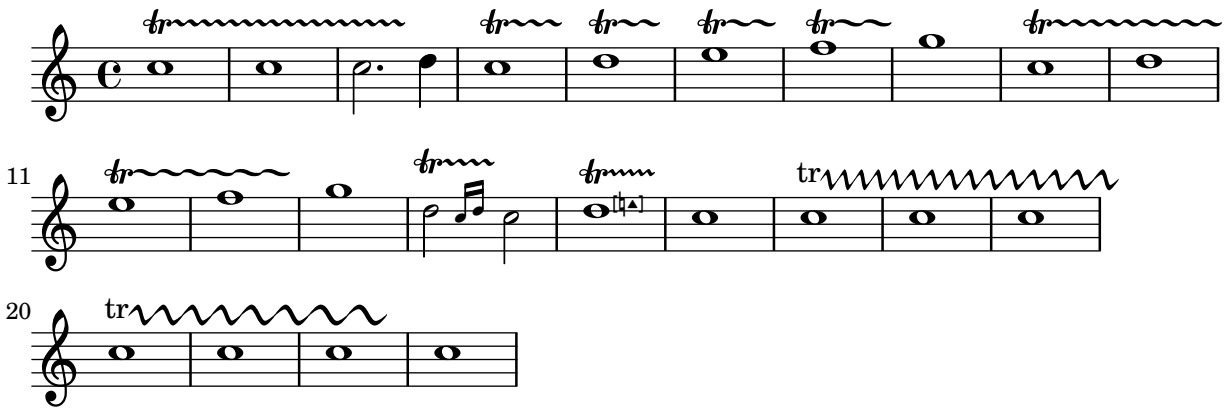
  c1 \ekmStartTrillSpan #-1
  d1 \ekmStartTrillSpan #-2
  e1 \ekmStartTrillSpan #-3
  f1 \ekmStartTrillSpan #-4
  g1 \stopTrillSpan

  c,1 \ekmStartTrillSpan #'(-1 . -4)
  d \break e f g \stopTrillSpan

  \afterGrace
  d2 \ekmStartTrillSpan #1
  { c16[ d] }
  c2 \stopTrillSpan

  \ekmPitchedTrill #'triangle #'bracket
  d1 \ekmStartTrillSpan #4
  e
  c \stopTrillSpan

  c1 \ekmStartSpan
       #'trill-vibrato-large
       #'(3 . -3)
       #"tr"
  c c \break c c c c \stopTrillSpan
}
```



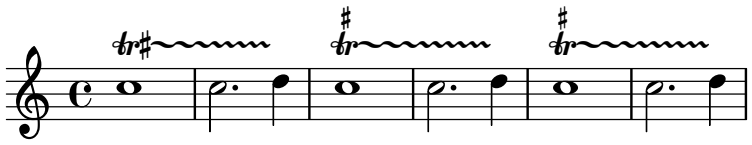
Draws a trill with a SMuFL accidental in three ways.

```
ekmUse = #'(trill)
\include "cosmufl.ily"

\relative c'' {
  % use markup
  c1 \ekmStartSpan
      #'trill
      #'(-3 . 2)
      #`(,(markup #:concat
            (#:ekm-char #xE566
              #:general-align Y -0.7
                #:fontsize -2
                  #:ekm-char #xE262))
          . 0)
  c2. d4 \stopTrillSpan

  % use a ligature
  c1 \ekmStartSpan
      #'trill
      #'(-3 . 2)
      #'((#xE262 #xE566) . 0)
  c2. d4 \stopTrillSpan

  % use a Cosmuflily command
  c1 \ekmStartTrillSpanAccidental
      #'(-3 . 2)
      #1/2
  c2. d4 \stopTrillSpan
}
```



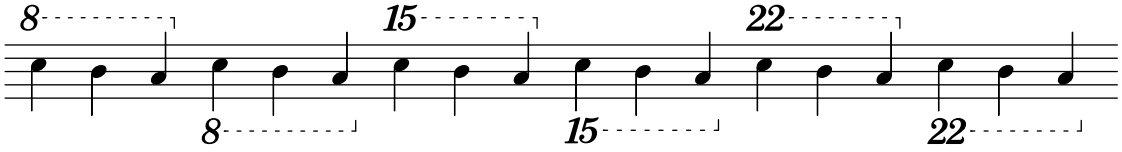
Ottavation

The ottavation style can be set with

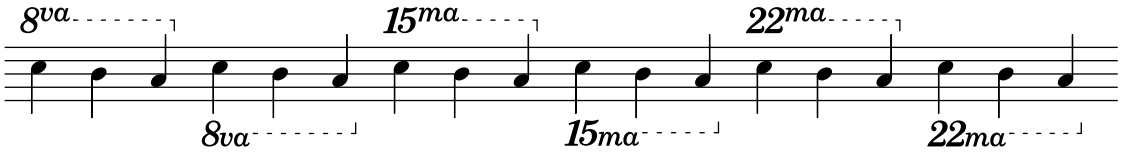
```
\set Staff.ottavationMarkups = #(ekm-ottavation STYLE)
```

STYLE

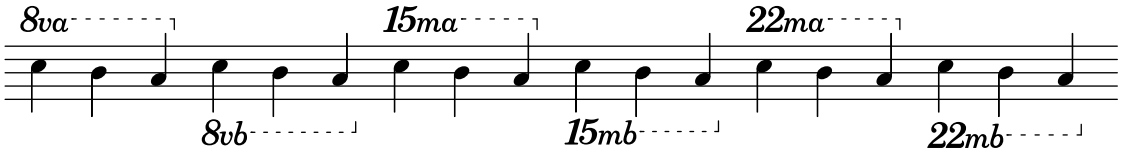
'numbers	±1	U+E510	ottava
	±2	U+E514	quindicesima
	±3	U+E517	ventiduesima



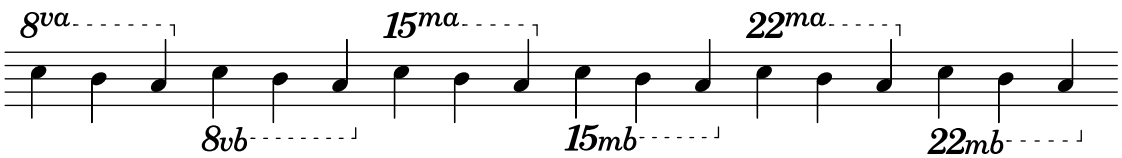
'ordinals	1	U+E511	ottavaAlta
	-1	U+E512	ottavaBassa
	2	U+E515	quindicesimaAlta
	-2	U+E516	quindicesimaBassa
	3	U+E518	ventiduesimaAlta
	-3	U+E519	ventiduesimaBassa



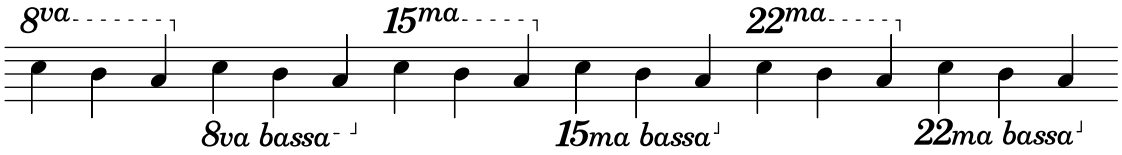
'simple-ordinals	1	U+E510	ottava
		U+EC97	octaveBaselineV
		U+EC91	octaveBaselineA
	-1	U+E51C	ottavaBassaVb
	2	U+E514	quindicesima
		U+EC95	octaveBaselineM
		U+EC91	octaveBaselineA
	-2	U+E51D	quindicesimaBassaMb
	3	U+E517	ventiduesima
		U+EC95	octaveBaselineM
		U+EC91	octaveBaselineA
	-3	U+E51E	ventiduesimaBassaMb



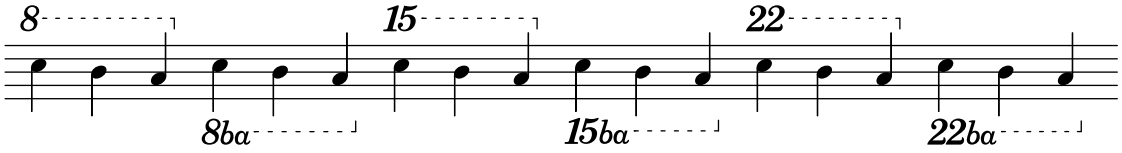
'ordinals-b	1	U+E511	ottavaAlta
	-1	U+E51C	ottavaBassaVb
	2	U+E515	quindicesimaAlta
	-2	U+E51D	quindicesimaBassaMb
	3	U+E518	ventiduesimaAlta
	-3	U+E51E	ventiduesimaBassaMb



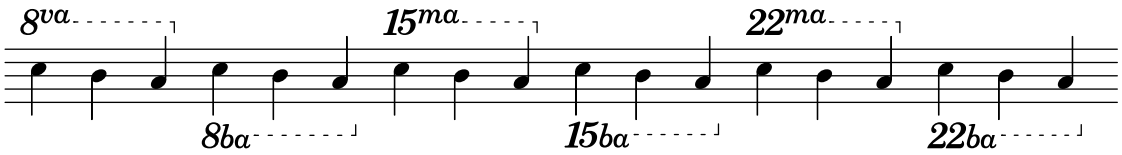
'ordinals-bassa	1	U+E511	ottavaAlta
	-1	U+E512	ottavaBassa
		U+E51F	octaveBassa
	2	U+E515	quindicesimaAlta
	-2	U+E516	quindicesimaBassa
		U+E51F	octaveBassa
	3	U+E518	ventiduesimaAlta
	-3	U+E519	ventiduesimaBassa
		U+E51F	octaveBassa



'numbers-ba	1	U+E510	ottava
	-1	U+E513	ottavaBassaBa
	2	U+E514	quindicesima
	-2	U+E514	quindicesima
		U+EC93	octaveBaselineB
		U+EC91	octaveBaselineA
	3	U+E517	ventiduesima
	-3	U+E517	ventiduesima
		U+EC93	octaveBaselineB
		U+EC91	octaveBaselineA
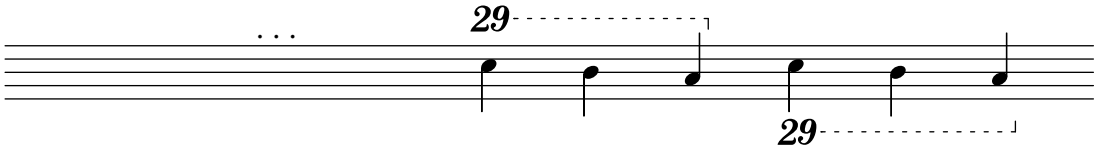


'ordinals-ba	1	U+E511	ottavaAlta
	-1	U+E513	ottavaBassaBa
	2	U+E515	quindicesimaAlta
	-2	U+E514	quindicesima
		U+EC93	octaveBaselineB
		U+EC91	octaveBaselineA
	3	U+E518	ventiduesimaAlta
	-3	U+E517	ventiduesima
		U+EC93	octaveBaselineB
		U+EC91	octaveBaselineA
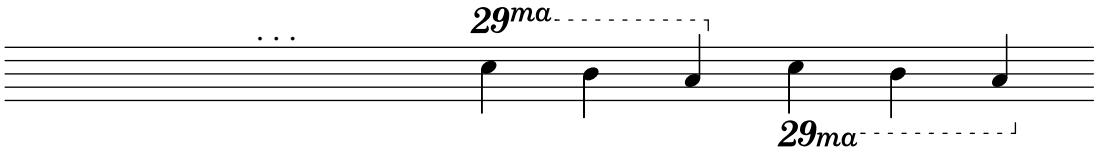


> According to the implementation notes of SMuFL Octaves, the suffixes *vb* and *mb* as in `'simple-ordinals` and `'ordinals-b` are corruptions of the more correct forms *va bassa* and *ma bassa* as in `'ordinals-bassa`. The recommended abbreviation for *8va bassa* is *8ba* as in `'numbers-ba` and `'ordinals-ba`.

with Ekmelos
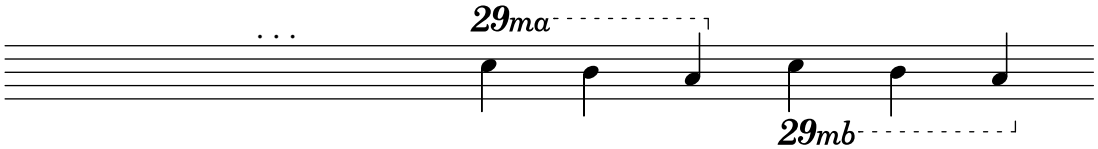
'numbers
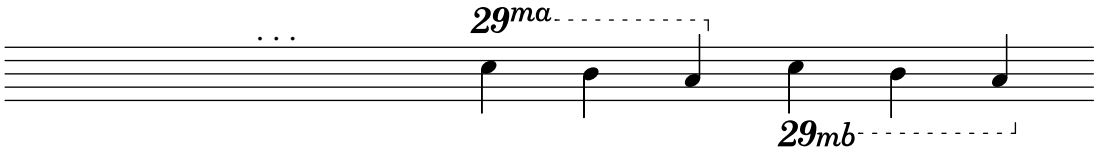


'ordinals
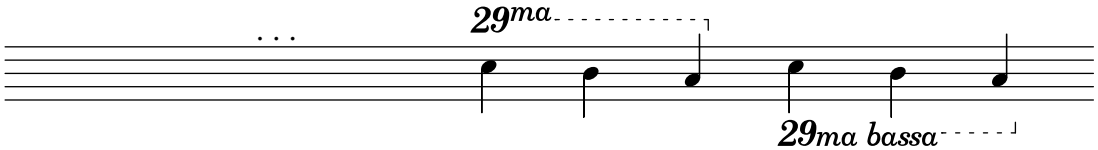


'simple-ordinals
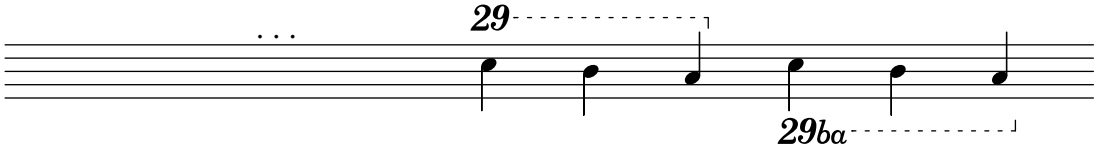


'ordinals-b
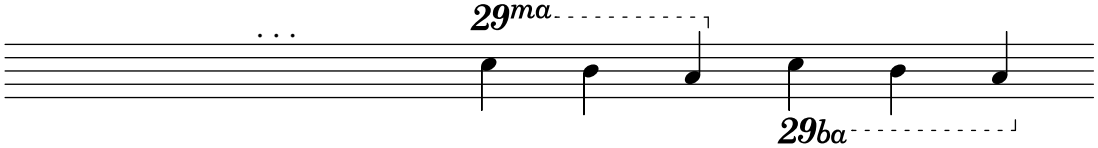


'ordinals-bassa



'numbers-ba



'ordinals-ba



`\ekm-ottavation DEFINITION`

Draw an ottavation text according to DEFINITION as markup.

`8`		U+E510	ottava
`8^va`		U+E511	ottavaAlta
`8va`		U+E512	ottavaBassa
`8ba`		U+E513	ottavaBassaBa
`8vb`		U+E51C	ottavaBassaVb
`15`		U+E514	quindicesima
`15^ma`		U+E515	quindicesimaAlta
`15ma`		U+E516	quindicesimaBassa
`15mb`		U+E51D	quindicesimaBassaMb
`22`		U+E517	ventiduesima
`22^ma`		U+E518	ventiduesimaAlta
`22ma`		U+E519	ventiduesimaBassa
`22mb`		U+E51E	ventiduesimaBassaMb
`(`		U+E51A	octaveParensLeft
`)`		U+E51B	octaveParensRight
`bassa`		U+E51F	octaveBassa
`loco`		U+EC90	octaveLoco
`^a`		U+EC92	octaveSuperscriptA
`^b`		U+EC94	octaveSuperscriptB
`^m`		U+EC96	octaveSuperscriptM
`^v`		U+EC98	octaveSuperscriptV
`a`		U+EC91	octaveBaselineA
`b`		U+EC93	octaveBaselineB
`m`		U+EC95	octaveBaselineM
`v`		U+EC97	octaveBaselineV

with Ekmelos

`8^vb`		U+F652	ottavaBassaSupVb
`15^mb`		U+F653	quindicesimaBassaSupMb
`22^mb`		U+F654	ventiduesimaBassaSupMb
`29`		U+F6F8	ventinovesima
`29^ma`		U+F6F9	ventinovesimaAlta
`29ma`		U+F6FA	ventinovesimaBassa
`29^mb`		U+F655	ventinovesimaBassaSupMb
`29mb`		U+F6FB	ventinovesimaBassaMb

`\ekmSmuflOn #'tuplet`

Draw SMuFL tuplet numbers as numerator only. Set the first formatting function below, so this switch is not required if one of these functions is set explicitly.

:	:	U+E88A	tupletColon

See `\ekm-number` for tuplet digits.

Tuplet formatting functions

```
ekm-tuplet-number::calc-denominator-text

ekm-tuplet-number::calc-fraction-text

(ekm-tuplet-number::non-default-tuplet-denominator-text NUM)

(ekm-tuplet-number::non-default-tuplet-fraction-text NUM DENOM)

(ekm-tuplet-number::append-note-wrapper
    FUNCTION DURATION)

(ekm-tuplet-number::fraction-with-notes
    NUM-DURATION DENOM-DURATION)

(ekm-tuplet-number::non-default-fraction-with-notes
    NUM NUM-DURATION DENOM DENOM-DURATION)
```

The last three functions draw `'metronome` style notes for the specified durations.

`(ekm-tuplet-number NUM DENOM)`

Draw NUM:DENOM, or NUM only if DENOM is 0. Use the actual tuplet fraction for NUM or DENOM if `#f` is specified. It is called by the first four functions above, i.e. they are equivalent to:

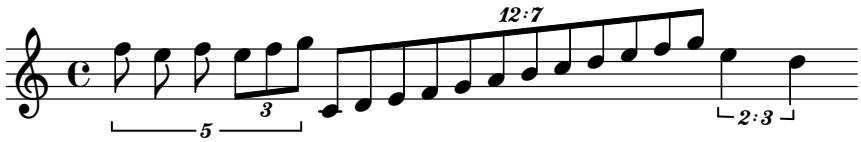
```
(ekm-tuplet-number #f 0)
(ekm-tuplet-number #f #f)
(ekm-tuplet-number NUM 0)
(ekm-tuplet-number NUM DENOM)
```

Examples

```
\relative c'' {
  \cadenzaOn

  \override TupletNumber.text = #ekm-tuplet-number::calc-denominator-text
  \tuplet 5/4 {
    f8 e f
    \tuplet 3/2 { e[ f g] }
  }

  \override TupletNumber.text = #ekm-tuplet-number::calc-fraction-text
  \tuplet 12/7 { c,,8[ d e f g a b c d e f g] }
  \tuplet 2/3 { e4 d }
}
```
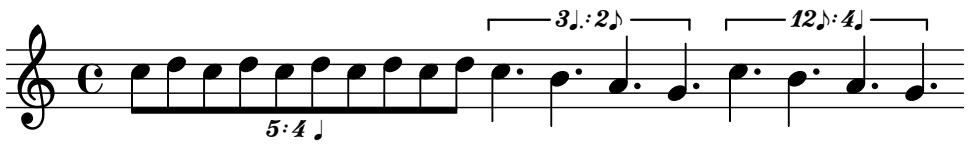



```
\relative c'' {
  \cadenzaOn

  \once \override TupletNumber.text =
    #(ekm-tuplet-number::append-note-wrapper
       ekm-tuplet-number::calc-fraction-text
       (ly:make-duration 2 0))
  \tuplet 5/4 { c8[ d c d c d c d c d] }

  \once \override TupletNumber.text =
    #(ekm-tuplet-number::fraction-with-notes
      (ly:make-duration 2 1)
      (ly:make-duration 3 0))
  \tuplet 3/2 { c4. b a g }

  \once \override TupletNumber.text =
    #(ekm-tuplet-number::non-default-fraction-with-notes
      12 (ly:make-duration 3 0)
      4 (ly:make-duration 2 0))
  \tuplet 3/2 { c4. b a g }
}
```



`\ekmSmuflOn #'fingering`

Draw SMuFL fingering instructions specified with a digit, with `\finger`, or with `\thumb`, as well as right-hand fingerings specified with `\rightHandFinger`, all using `\ekm-finger`.

`\thumb` is equivalent to `\finger "th"`

`\ekm-finger DEFINITION`

Draw a fingering instruction according to DEFINITION as markup.
If DEFINITION has the prefix `*` the italic style of some symbols (digits and brackets) is drawn.

(	(	U+ED28	fingeringLeftParenthesis
)	)	U+ED29	fingeringRightParenthesis
[	[	U+ED2A	fingeringLeftBracket
]	]	U+ED2B	fingeringRightBracket
.	•	U+ED2C	fingeringSeparatorMiddleDot
'	◦	U+ED2D	fingeringSeparatorMiddleDotWhite
/	/	U+ED2E	fingeringSeparatorSlash
~~	⁀	U+ED20	fingeringSubstitutionAbove
~	‿	U+ED21	fingeringSubstitutionBelow
–	-	U+ED22	fingeringSubstitutionDash
M	[	U+ED23	fingeringMultipleNotes
th	0	U+E624	stringsThumbPosition
ht	ɔ	U+E625	stringsThumbPositionTurned
T	T	U+ED16	fingeringTUpper
t	*t*	U+ED18	fingeringTLower
p	*p*	U+ED17	fingeringPLower
i	*i*	U+ED19	fingeringILower
m	*m*	U+ED1A	fingeringMLower
a	*a*	U+ED1B	fingeringALower
c	*c*	U+ED1C	fingeringCLower
x	*x*	U+ED1D	fingeringXLower
e	*e*	U+ED1E	fingeringELower
o	*o*	U+ED1F	fingeringOLower
q	*q*	U+ED8E	fingeringQLower
s	*s*	U+ED8F	fingeringSLower
R	⌊	U+E66E	keyboardPlayWithRH
RE	⌋	U+E66F	keyboardPlayWithRHEnd
L	⌈	U+E670	keyboardPlayWithLH
LE	⌉	U+E671	keyboardPlayWithLHEnd

See `\ekm-number` for fingering digits.

`\ekmPlayWith HAND DIR MUSIC`

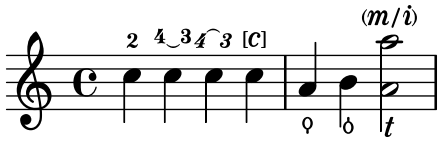
Draw the symbol for the fingering token `R`, `RE`, `L`, or `LE` (see above) alongside the notes in MUSIC.
HAND is `RIGHT` or `LEFT`.
DIR is `START` (`LEFT`) for the start symbol placed to the left, or `STOP` (`RIGHT`) for the end symbol placed to the right.

Examples

```
\relative c'' {
  \ekmSmuflOn #'fingering

  c4 - 2
  c - \finger "4~3"
  c - \finger "*4~~3"
  c - \finger "[c]"
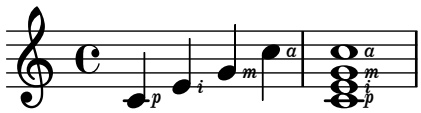
  a _ \finger "th"
  b _ \finger "ht"
  < a - \finger "t"
    a' - \finger "(m_/_i)" >2
}
```




```
\relative c' {
  \ekmSmuflOn #'fingering

  c \rightHandFinger #1
  e \rightHandFinger #2
  g \rightHandFinger #3
  c \rightHandFinger #4
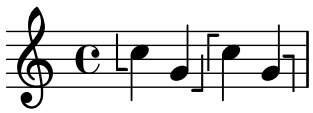
  < c, \rightHandFinger #1
    e  \rightHandFinger #2
    g  \rightHandFinger #3
    c  \rightHandFinger #4 >1
}
```



```
\relative c'' {
  \ekmSmuflOn #'fingering

  \ekmPlayWith #RIGHT #START c
  \ekmPlayWith #RIGHT #STOP g

  \ekmPlayWith #LEFT #START c
  \ekmPlayWith #LEFT #STOP g
}
```



String Number Indications

`\ekmSmuflOn #'stringnumber`

Draw SMuFL string number indications specified with \NUMBER, using `\ekm-string-number` .

`\ekm-string-number ARG`

Draw a string number indication as markup.
ARG is a number or string. For a number or a string representing a number, either the precomposed symbol or the number according to `style` with a circle around is drawn. Any other string, e.g. a Roman numeral, is drawn in italic style.
Used properties:

- `style` ('sans) , any number style except `'string`.
- `size` (0) relative to the font size for numbers.

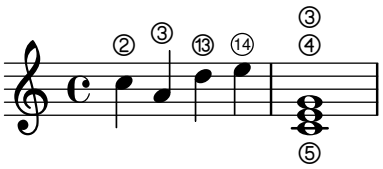
See `\ekm-number` for string number (guitar string) symbols and for number styles.

Examples

```
\relative c'' {
  \ekmSmuflOn #'stringnumber

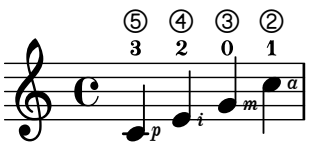
  c \2
  a \3
  d \13
  e \14

  < c,\5 e\4 g\3 >1
}
```



```
\relative c' {
  \ekmSmuflOn #'(fingering stringnumber)

  < c -3 \5 \rightHandFinger #1 >
  < e -2 \4 \rightHandFinger #2 >
  < g -0 \3 \rightHandFinger #3 >
  < c -1 \2 \rightHandFinger #4 >
}
```



Piano Pedals

`\ekmSmuflOn #'pedal`

Draw SMuFL piano pedals for sustain, sostenuto, and una corda, using `\ekm-piano-pedal` .

`\ekm-piano-pedal DEFINITION`

Draw piano pedal symbols according to DEFINITION as markup.

`Ped.`		U+E650	keyboardPedalPed
`P`		U+E651	keyboardPedalP
`e`		U+E652	keyboardPedalE
`d`		U+E653	keyboardPedalD
`Sost.`		U+E659	keyboardPedalSost
`S`		U+E65A	keyboardPedalS
`.`		U+E654	keyboardPedalDot
`-`		U+E658	keyboardPedalHyphen
`*`		U+E655	keyboardPedalUp
`o`		U+E65D	keyboardPedalUpSpecial
`,`		U+E65B	keyboardPedalHalf2
`'`		U+E65C	keyboardPedalHalf3
`H`		U+E656	keyboardPedalHalf
`^`		U+E657	keyboardPedalUpNotch
`l`		U+E65E	keyboardLeftPedalPictogram
`m`		U+E65F	keyboardMiddlePedalPictogram
`r`		U+E660	keyboardRightPedalPictogram
`(`		U+E676	keyboardPedalParensLeft
`)`		U+E677	keyboardPedalParensRight

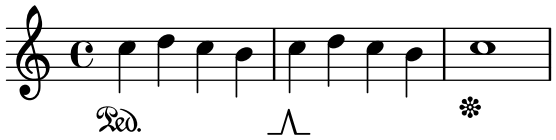
with Ekmelos

`Ped`		U+F434	keyboardPedalPedNoDot
`ConPed.`		U+F79E	keyboardPedalConPed
`SenzaPed.`		U+F79F	keyboardPedalSenzaPed
`Sost`		U+F435	keyboardPedalSostNoDot
`Sos.`		U+F6D1	keyboardPedalSos2
`sos.`		U+F6D0	keyboardPedalSos
`unacorda`		U+F6CC	keyboardPedalUnaCorda
`trecorde`		U+F6CD	keyboardPedalTreCorde
`u.c.`		U+F6CE	keyboardPedalUC
`t.c.`		U+F6CF	keyboardPedalTC
`1/2Ped`		U+F6B0	keyboardPedalHalf4
`|1/4`		U+F6BA	keyboardPedalPosQuarter
`|1/2`		U+F6BB	keyboardPedalPosHalf
`|3/4`		U+F6BC	keyboardPedalPosThreeQuarters
`|1`		U+F6BD	keyboardPedalPosFull

Examples
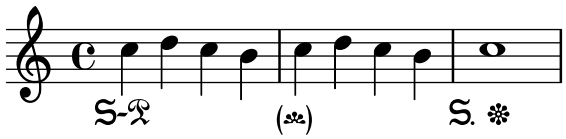
```
\new Staff \with {
  \ekmSmuflOn #'pedal
}
\relative c'' {
  \set Staff.pedalSustainStrings = #'("Ped." "H" "*")

  c4 \sustainOn d c b
  c \sustainOff \sustainOn d c b
  c1 \sustainOff
}
```



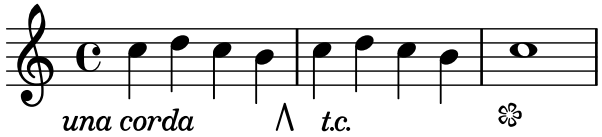

```
\new Staff \with {
  \ekmSmuflOn #'pedal
}
\relative c'' {
  \set Staff.pedalSostenutoStyle = #'text
  \set Staff.pedalSostenutoStrings = #'("S-P" "(')" "S. *")

  c4 \sostenutoOn d c b
  c \sostenutoOff \sostenutoOn d c b
  c1 \sostenutoOff
}
```




```
\new Staff \with {
  \ekmSmuflOn #'pedal
}
\relative c'' {
  % draws Ekmelos glyphs "unacorda" and "t.c."
  \set Staff.pedalUnaCordaStyle = #'text
  \set Staff.pedalUnaCordaStrings = #'("unacorda" "^___t.c." "o")

  c4 \unaCorda d c b
  c \treCorde \unaCorda d c b
  c1 \treCorde
}
```



Harp Pedals

`\ekm-harp-pedal DEFINITION`

Draw a harp pedal diagram according to DEFINITION as markup. Space between tokens is ignored.

^		U+E680	harpPedalRaised
o^			
–		U+E681	harpPedalCentered
o–			
v		U+E682	harpPedalLowered
ov			
|		U+E683	harpPedalDivider

with Ekmelos

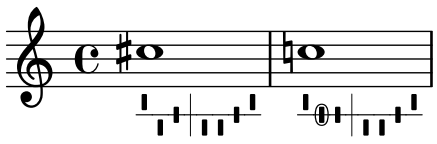
o^		U+F648	harpPedalRaisedChange
o–		U+F649	harpPedalCenteredChange
ov		U+F64A	harpPedalLoweredChange

`\ekm-harp-change EXTEXT Y`

Draw EXTEXT as markup with an ellipse translated vertically by Y. Used by change tokens such as `o^`.

Examples

```
\relative c'' {
  \textLengthOn
  cis1 _ \markup \ekm-harp-pedal #"^ v – | v v – ^"
  c! _ \markup \ekm-harp-pedal #"^ o– – | v v – ^"
}
```



Fret Diagrams

`\ekm-fret-diagram-terse DEFINITION`

Draw a fret diagram according to DEFINITION as markup. Fingering is always placed below.
Used properties:

- `fret-diagram-details top-fret-thickness` (3): A value > 1 draws the …Nut glyph.
- `fret-diagram-details finger-code` (#t) : `'none` draws no fingering.
- `fret-diagram-details finger-style` (`'sans`)

3 strings		U+E850	fretboard3String
		U+E851	fretboard3StringNut
4 strings		U+E852	fretboard4String
		U+E853	fretboard4StringNut
5 strings		U+E854	fretboard5String
		U+E855	fretboard5StringNut
6 strings		U+E856	fretboard6String
		U+E857	fretboard6StringNut
.		U+E858	fretboardFilledCircle
x		U+E859	fretboardX
o		U+E85A	fretboardO

Examples

```
\relative c'' {
  \textLengthOn

  c1 ^ \markup \ekm-fret-diagram-terse #"x;3-3;2-2;o;1-1;o;"

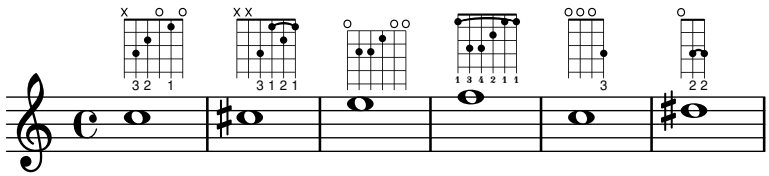
  cis1 ^ \markup \ekm-fret-diagram-terse #"x;x;3-3;1-1-(;2-2;1-1-);"

  e1 ^ \markup {
       \override #'(fret-diagram-details . (
                     (top-fret-thickness . 1)
                     (finger-code . none)))
       \ekm-fret-diagram-terse #"o;2-2;2-3;1-1;o;o;"
     }

  f1 ^ \markup {
       \override #'(fret-diagram-details . (
                     (finger-style . fingering)))
       \ekm-fret-diagram-terse #"1-1-(;3-3;3-4;2-2;1-1;1-1-);"
     }

  c1 ^ \markup \ekm-fret-diagram-terse #"o;o;o;3-3;"

  dis1 ^ \markup \ekm-fret-diagram-terse #"o;3-2-(;3-2-);"
}
```



Accordion Registers

`\ekm-accordion NAME`

Draw an accordion register symbol as markup. Most of the symbols use precomposed glyphs. The others are composed using accdnCombRH3RanksEmpty (U+E8C6) etc.
NAME (a string) has an optional prefix for the register style separated by a space. Default is `d` (discant).

> The module `(scm accreg)` is not required.

`\ekmAccordion NAME`

Set an accordion register symbol as a standalone music event. This is equivalent to

```
<> ^ \markup \ekm-accordion NAME
```

NAME

`"d …"`	Discant		
`"d 1"`		U+E8A4	accdnRH3RanksBassoon
`"d 10"`		U+E8A1	accdnRH3RanksClarinet
`"d 11"`		U+E8AB	accdnRH3RanksBandoneon
`"d 1+0"`		U+E8A2	accdnRH3RanksUpperTremolo8
`"d 1+1"`			
`"d 1-0"`		U+E8A3	accdnRH3RanksLowerTremolo8
`"d 1-1"`			
`"d 20"`		U+E8AE	accdnRH3RanksTwoChoirs
`"d 21"`		U+E8AF	accdnRH3RanksTremoloLower8ve
`"d 2+0"`		U+E8A6	accdnRH3RanksViolin
`"d 2+1"`		U+E8AC	accdnRH3RanksAccordion
`"d 2-0"`			
`"d 2-1"`			
`"d 30"`		U+E8A8	accdnRH3RanksAuthenticMusette
`"d 31"`		U+E8B1	accdnRH3RanksDoubleTremoloLower8ve
`"d 100"`		U+E8A0	accdnRH3RanksPiccolo
`"d 101"`		U+E8A9	accdnRH3RanksOrgan
`"d 110"`		U+E8A5	accdnRH3RanksOboe
`"d 111"`		U+E8AA	accdnRH3RanksHarmonium
`"d 11+0"`			
`"d 11+1"`			
`"d 11-0"`			
`"d 11-1"`			

"d 120"		U+E8B0	accdnRH3RanksTremoloUpper8ve
"d 121"		U+E8AD	accdnRH3RanksMaster
"d 12+0"		U+E8A7	accdnRH3RanksImitationMusette
"d 12+1"			
"d 12-0"			
"d 12-1"			
"d 130"		U+E8B2	accdnRH3RanksDoubleTremoloUpper8ve
"d 131"		U+E8B3	accdnRH3RanksFullFactory

"sb …" Standard bass

"sb Soprano"		U+E8B4	accdnRH4RanksSoprano
"sb Alto"		U+E8B5	accdnRH4RanksAlto
"sb Tenor"		U+E8B6	accdnRH4RanksTenor
"sb Master"		U+E8B7	accdnRH4RanksMaster
"sb Soft Bass"		U+E8B8	accdnRH4RanksSoftBass
"sb Soft Tenor"		U+E8B9	accdnRH4RanksSoftTenor
"sb Bass/Alto"		U+E8BA	accdnRH4RanksBassAlto

"sb4 …" Standard bass, four reed

"sb4 Soprano"		U+E8B4	accdnRH4RanksSoprano
"sb4 Alto"		U+E8B5	accdnRH4RanksAlto
"sb4 Tenor"			
"sb4 Master"			
"sb4 Soft Bass"			
"sb4 Bass/Alto"		U+E8BA	accdnRH4RanksBassAlto
"sb4 Soft Bass/Alto"			
"sb4 Soft Tenor"		U+E8B9	accdnRH4RanksSoftTenor

"sb5 …"	Standard bass, five reed		
"sb5 Bass/Alto"		U+E8BA	accdnRH4RanksBassAlto
"sb5 Soft Bass/Alto"			
"sb5 Alto"			
"sb5 Tenor"			
"sb5 Master"			
"sb5 Soft Bass"			
"sb5 Soft Tenor"		U+E8B9	accdnRH4RanksSoftTenor
"sb5 Soprano"		U+E8B4	accdnRH4RanksSoprano
"sb5 Sopranos"			
"sb5 Solo Bass"			

"sb6 …"	Standard bass, six reed		
"sb6 Soprano"		U+E8B4	accdnRH4RanksSoprano
"sb6 Alto"			
"sb6 Soft Tenor"		U+E8B9	accdnRH4RanksSoftTenor
"sb6 Master"		U+E8B7	accdnRH4RanksMaster
"sb6 Alto/Soprano"			
"sb6 Bass/Alto"		U+E8BA	accdnRH4RanksBassAlto
"sb6 Soft Bass"		U+E8B8	accdnRH4RanksSoftBass

"fb …"	Free bass		
"fb 10"		U+E8BB	accdnLH2Ranks8Round
"fb 1"		U+E8BC	accdnLH2Ranks16Round
"fb 11"		U+E8BD	accdnLH2Ranks8Plus16Round
"fb Master"		U+E8BE	accdnLH2RanksMasterRound
"fb Master 1"		U+E8BF	accdnLH2RanksMasterPlus16Round
"fb Master 11"		U+E8C0	accdnLH2RanksFullMasterRound

"sq …"	Square		
"sq 1"		U+E8C1	accdnLH3Ranks8Square
"sq 100"		U+E8C2	accdnLH3Ranks2Square
"sq 2"		U+E8C3	accdnLH3RanksDouble8Square
"sq 101"		U+E8C4	accdnLH3Ranks2Plus8Square
"sq 102"		U+E8C5	accdnLH3RanksTuttiSquare

Accordion Ricochet

`\ekmRicochet NUMBER`

Draw a ricochet symbol as an expressive mark (script). [LP]

NUMBER

2		U+E8CD	accdnRicochet2
3		U+E8CE	accdnRicochet3
4		U+E8CF	accdnRicochet4
5		U+E8D0	accdnRicochet5
6		U+E8D1	accdnRicochet6

`\ekmStemRicochet NUMBER MUSIC`

Draw a ricochet symbol vertically centered on the stems in MUSIC.

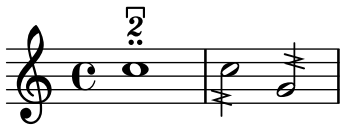
NUMBER

2		U+E8D2	accdnRicochetStem2
3		U+E8D3	accdnRicochetStem3
4		U+E8D4	accdnRicochetStem4
5		U+E8D5	accdnRicochetStem5
6		U+E8D6	accdnRicochetStem6

Examples

```
\relative c'' {
  \ekmSmuflOn #'script
  c1 ^ \ekmRicochet #2

  \ekmStemRicochet #3 { c2 g }
}
```



`\ekmSmuflOn #'fbass`

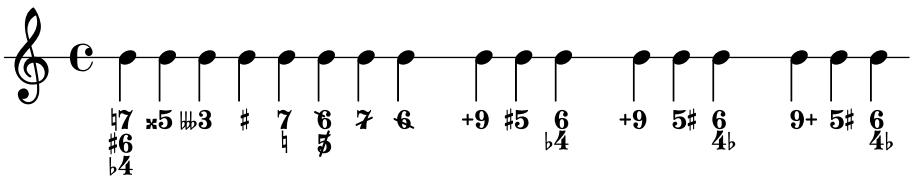
Draw SMuFL bass figures with `\figuremode`. Some raised / diminished figures use precomposed glyphs which ignore the property `figuredBassPlusDirection`.

!	♮	U+EA65	figbassNatural
-	♭	U+EA64	figbassFlat
+	♯	U+EA66	figbassSharp
--	𝄫	U+EA63	figbassDoubleFlat
++	𝄪	U+EA67	figbassDoubleSharp
---	𝄫♭	U+ECC1	figbassTripleFlat
+++	𝄪♯	U+ECC2	figbassTripleSharp
\+	+	U+EA6C	figbassPlus
/	⁄	U+EA6D	figbassCombiningRaising
\\	⧵	U+EA6E	figbassCombiningLowering
2\+	2+	U+EA53	figbass2Raised
4\+	4+	U+EA56	figbass4Raised
5\+	5+	U+EA58	figbass5Raised1
5\\	5⧵	U+EA59	figbass5Raised2
5/	5⁄	U+EA5A	figbass5Raised3
6\\	6⧵	U+EA5C	figbass6Raised
6\+	6+	U+EA6F	figbass6Raised2
7\+	7+	U+EA5E	figbass7Raised1
7\\	7⧵	U+EA5F	figbass7Raised2
7/	7⁄	U+ECC0	figbass7Diminished
9\\	9⧵	U+EA62	figbass9Raised

See `\ekm-number` for figured bass digits.

Examples

```
\new Staff
<<
  \relative c'' {
    \cadenzaOn
    b4 b b b b b b b s
    b b b s
    b b b s
    b b b
  }
  \figures {
    <7! 6+ 4->  <5++>  <3--->  <_+>  <7 _!>  <6\+ 5/>  <7/>  <6\\> r
    <9\+>  <5+>  <6 4->  r
    \set figuredBassAlterationDirection = #RIGHT
    <9\+>  <5+>  <6 4->  r
    \set figuredBassPlusDirection = #RIGHT
    <9\+>  <5+>  <6 4->
  }
>>
\layout {
  \context {
    \Score
    \ekmSmuflOn #'fbass
    \override StaffSymbol.line-count = #1
  }
}
```



`\ekmSmuflOn #'lyric`

Draw the words in a lyric input mode (`\lyricmode` etc.) with `\ekm-tied-lyric`.
Note that the characters _ and `%` must be quoted in order to be passed on to this command.
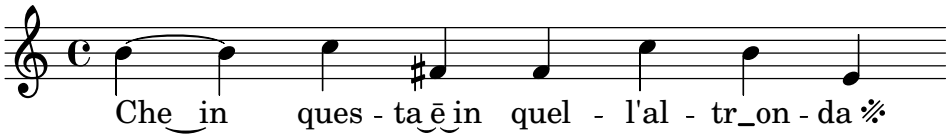
`\ekm-tied-lyric STRING`

Draw STRING as markup, replacing tokens with the corresponding glyphs. The space between the adjoining words depends on the width of the respective glyph, while the property `word-space` is ignored.
In the token `~?~` for narrow elision, `?` can be any Unicode character, not only ASCII.

~	‿	U+E551	lyricsElision
~?~	‿	U+E550	lyricsElisionNarrow
~~	‿	U+E552	lyricsElisionWide
_	–	U+E553	lyricsHyphenBaseline
%	𝄎	U+E555	lyricsTextRepeat

Example

```
\relative {
  \cadenzaOn
  b'~ b c fis, fis c' b e,
}
\addlyrics {
  Che~~in ques -- ta~ē~in quel -- l'al -- "tr_on" -- "da %"
}
\layout {
  \context {
    \Score
    \ekmSmuflOn #'lyric
  }
}
```



Chord Name Symbols

`\ekmSmuflOn #'chord`

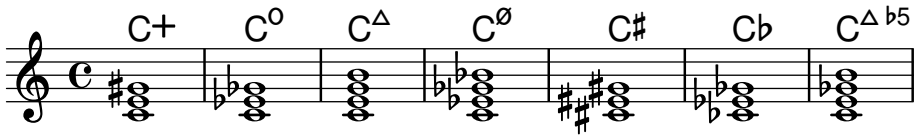
Draw SMuFL chord name symbols.

`dim`	o	U+E870	csymDiminished
`m7.5-`	ø	U+E871	csymHalfDiminished
`maj7`	△	U+E873	csymMajorSeventh
`aug`	+	U+E872	csymAugmented
`-`	−	U+E874	csymMinor

Examples

```
someChords = \chordmode {
  c1:aug
  c:dim
  c:maj7
  c:m7.5-
  % use cosmufl.ily to draw SMuFL accidentals
  cis
  ces
  <c' e' ges' b'>
}

<<
  \new ChordNames { \someChords }
  \someChords
>>
\layout {
  \context {
    \Score
    \ekmSmuflOn #'chord
  }
}
```



`\ekm-analytics DEFINITION`

Draw analytics symbols according to DEFINITION as markup.

`H`		U+E860	analyticsHauptstimme
`CH`		U+E86A	analyticsChoralmelodie
`RH`		U+E86B	analyticsHauptrhythmus
`N`		U+E861	analyticsNebenstimme
`[`		U+E862	analyticsStartStimme
`]`		U+E863	analyticsEndStimme
`Th`		U+E864	analyticsTheme
`hT`		U+E865	analyticsThemeRetrograde
`ihT`		U+E866	analyticsThemeRetrogradeInversion
`iTh`		U+E867	analyticsThemeInversion
`T`		U+E868	analyticsTheme1
`iT`		U+E869	analyticsInversion1

Function Theory Symbols

Markup Commands

`\ekm-func DEFINITION`

Draw a function theory symbol according to DEFINITION as markup. The definition string consists of several parts which are all optional:

BRACKET FUNCTION , BASS , SOPRANO ^ EXTRA ... BRACKET

The BASS symbol is placed below the FUNCTION symbol.
The SOPRANO symbol is placed above the FUNCTION symbol.
The EXTRA symbols are stacked vertically and raised to the right of the FUNCTION symbol.
A leading / trailing BRACKET `( [ { } ] )` is placed separately before / after the entire symbol.
Used properties:

- `font-size` (0) for FUNCTION.
- `func-size` (-6) for BASS, SOPRANO, and EXTRA.
- `func-skip` (1.6) Vertical space of BASS, SOPRANO, and EXTRA.
- `func-raise` (1.5) Raise of EXTRA.
- `func-space` (0.3) Horizontal space around FUNCTION.

`\ekm-func-column PIVOT DEFINITION-UP DEFINITION-DOWN`

Draw a centered column of two function theory symbols as markup.
PIVOT true draws a pivot chord symbol in between.
DEFINITION-UP/-DOWN is a string as with `\ekm-func` or a list of such definition strings.
A colon `:` in the string draws the part up to including the `:` with LilyPond's `sans` font.
DEFINITION-DOWN can be `#f` to draw a single symbol.
In case of two symbols, they are drawn with `\ekm-func`'s property `func-raise` set to 0 to avoid too high symbols when combined with a pivot.
Used properties:

- `font-size` (0)
- `y-padding` (1.6)

Music Functions

They are intended for use in a Lyrics context.

> The `Text_spanner_engraver` must be included to draw lines or pivot spans.

`\ekmFunc DEFINITION`

Set a function theory symbol or a column of two symbols as a music expression.
DEFINITION is a string as with `\ekm-func-column` or a sequence of two such definition strings in the form `"UP|DOWN"` for a column.
DEFINITION can have an additional suffix:

- `-` starts a line after the symbol.
- `~` starts a pivot span. Requires a column of two symbols.
- `.` stops a line or pivot span at the symbol.
- `!` draws a pivot. Requires a column of two symbols.
- `+` inserts the symbol between notes with `\set stanza` .
- `*` dito but with the font size of `\ekm-func` .

The font size relative to `\ekm-func` (except for suffix `*`) can be set with:

```
#(set! ekm:func-size SIZE)
```

The standard value is -4.

`\ekmFuncList DEFINITION-LIST`

Set a sequence of function theory symbols as music expressions.
DEFINITION-LIST is a list of DEFINITIONs as with `\ekmFunc`.

T		U+EA8B	functionTUpper
Tg			
Tp			
t		U+EA8C	functionTLower
D		U+EA7F	functionDUpper
/D			
Dp			
DD		U+EA81	functionDD
/DD		U+EA82	functionSlashedDD
d		U+EA80	functionDLower
S		U+EA89	functionSUpper
Sg			
Sp			
SS		U+EA7D	functionSSUpper
s		U+EA8A	functionSLower
ss		U+EA7E	functionSSLower
F		U+EA99	functionFUpper
G		U+EA83	functionGUpper
g		U+EA84	functionGLower
I		U+EA9A	functionIUpper
i		U+EA9B	functionILower
K		U+EA9C	functionKUpper
k		U+EA9D	functionKLower
L		U+EA9E	functionLUpper
l		U+EA9F	functionLLower
M		U+ED00	functionMUpper
m		U+ED01	functionMLower
N		U+EA85	functionNUpper
n		U+EA86	functionNLower
P		U+EA87	functionPUpper
p		U+EA88	functionPLower
r		U+ED03	functionRLower
V		U+EA8D	functionVUpper
v		U+EA8E	functionVLower

<		U+EA7A	functionLessThan
>		U+EA7C	functionGreaterThan
–		U+EA7B	functionMinus
+		U+EA98	functionPlus
o		U+EA97	functionRing
(		U+EA91	functionParensLeft
)		U+EA92	functionParensRight
[		U+EA8F	functionBracketLeft
]		U+EA90	functionBracketRight
{		U+EA93	functionAngleLeft
}		U+EA94	functionAngleRight
..		U+EA95	functionRepetition1
..+		U+EA96	functionRepetition2
b		U+ED60	csymAccidentalFlat
#		U+ED62	csymAccidentalSharp
bb		U+ED64	csymAccidentalDoubleFlat
x		U+ED63	csymAccidentalDoubleSharp
=		U+ED61	csymAccidentalNatural
?			

with Ekmelos

/D		U+F644	functionSlashedD

`b # bb x =` draw the standard accidentals for chord symbols.
`?` draws a space with the dimensions of functionZero (U+EA70). Useful for empty extra symbols.
See `\ekm-number` for function theory digits.

Examples

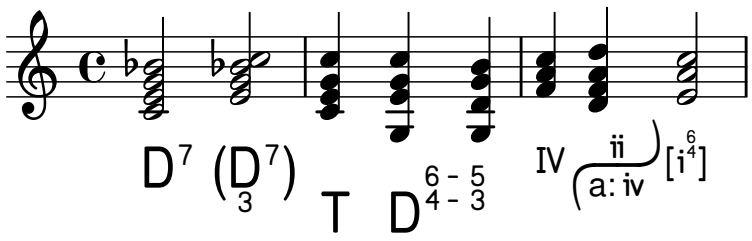
Use `\ekm-func` and `\ekm-func-column` in text scripts to attach function theory symbols to chords and spacer rest. It sets `\textLengthOn` and `TextScript.staff-padding` for a consistent vertical alignment.

```
\fixed c' {
  \textLengthOn

  \override TextScript.staff-padding = #6
  <c e g bes>2 _ \markup \ekm-func "D^7 "
  <e g bes! c'> _ \markup \ekm-func "(D,3^7)"

  \override TextScript.staff-padding = #9
  <c e g c'>4 _ \markup \ekm-func "T___"
  <g, e g c'> _ \markup \ekm-func "D^4^6"
  s _ \markup \ekm-func "^-^-"
  <g, d g b> _ \markup \ekm-func "^3^5"

  \override TextScript.staff-padding = #5
  \override TextScript.font-size = #-4
  <f a c'>4 _ \markup \ekm-func "IV"
  <d f a d'> _ \markup \ekm-func-column ##t "ii" "a:iv"
  <e a c'>2 _ \markup \ekm-func "[i^4^6]"
}
```



Example

Uses `\ekmFuncList` in a Lyrics context to synchronise function theory symbols to music. The context requires the `Text_spanner_engraver` and is aligned to a NullVoice context.
It is taken from lsr.di.unimi.it/LSR/Item?id=967 and adapted for Esmuflily.

```
funcSoprano = \relative c'' {
  e4 e e( d)
  c4 d d2
  d4 e8 d c4 c
  d8( c) <b g>4 c2
}

funcAltTenor = \relative c'' {
  <c g>4 <bes g> <a f>2
  <a d,>4 <c a> <c a>( <b g>)
  <b e,>2 <g e>4 <a f>
  <a d,>4 d,8( f) <g e>2
}

funcBass = \relative c {
  \clef bass
  c4 cis d2
  f4 fis g2
  gis2 bes4 a8 g
  fis4 g c,2
}

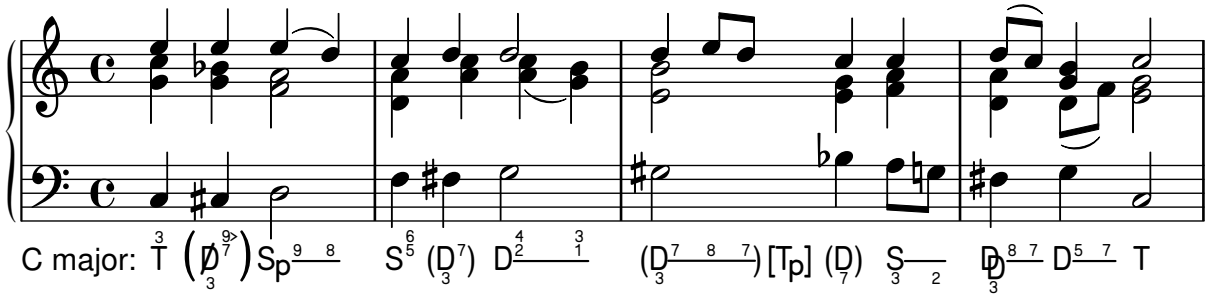
funcAligner = \relative c {
  c4 cis d d
  f4 fis g g
  gis4 gis8 gis bes4 a8 g
  fis8 fis g g c,2
}

funcSymbols = \lyricmode {
  \set stanza = #"C major:"
  \ekmFuncList #'(
    "T,,3"  " (*"  "/D,3^7^9>"  ")*"  "Sp^9-"  "^8."
    "S^5^6"  "(D,3^7)"  "D^2^4-"  "^1^3."
    "(D,3^7-"  "^8"  "^7."  "_) [Tp] +"  "(D,7)"  "S,3-"  " ,2."
    "DD,3^8-"  "^7."  "D^5-"  "^7."  "T"
  )
}

\layout {
  \context {
    \Lyrics
    \consists "Text_spanner_engraver"
    \override StanzaNumber.font-family = #'sans
    \override StanzaNumber.font-series = #'medium
  }
}
```

```
\new GrandStaff
<<
  \new Staff
    \new Voice \partCombine \funcSoprano \funcAltTenor

  \new Staff
  <<
    \new Voice \funcBass
    \new NullVoice = "funcaligner" \funcAligner
    \new Lyrics \lyricsto "funcaligner" \funcSymbols
  >>
>>
```



`\ekm-beater STYLE ORIENTATION`

Draw a percussion beater of STYLE according to ORIENTATION as markup.

Percussion beaters can have glyphs for several orientations. The remaining orientations are achieved by flipping or rotating through 90° or 30°. Most of the following styles have glyphs for the orientations `N,S,NE,NW` or `N,S`.

STYLE

`'xyl-soft`		U+E770	pictBeaterSoftXylophoneUp
`'xyl-medium`		U+E774	pictBeaterMediumXylophoneUp
`'xyl-hard`		U+E778	pictBeaterHardXylophoneUp
`'xyl-wood`		U+E77C	pictBeaterWoodXylophoneUp
`'glsp-soft`		U+E780	pictBeaterSoftGlockenspielUp
`'glsp-hard`		U+E784	pictBeaterHardGlockenspielUp
`'timpani-soft`		U+E788	pictBeaterSoftTimpaniUp
`'timpani-medium`		U+E78C	pictBeaterMediumTimpaniUp
`'timpani-hard`		U+E790	pictBeaterHardTimpaniUp
`'timpani-wood`		U+E794	pictBeaterWoodTimpaniUp
`'yarn-soft`		U+E7A2	pictBeaterSoftYarnUp
`'yarn-medium`		U+E7A6	pictBeaterMediumYarnUp
`'yarn-hard`		U+E7AA	pictBeaterHardYarnUp
`'gum-soft`		U+E7BB	pictGumSoftUp
`'gum-medium`		U+E7BF	pictGumMediumUp
`'gum-hard`		U+E7C3	pictGumHardUp
`'bass-soft`		U+E798	pictBeaterSoftBassDrumUp
`'bass-medium`		U+E79A	pictBeaterMediumBassDrumUp
`'bass-hard`		U+E79C	pictBeaterHardBassDrumUp
`'bass-metal`		U+E79E	pictBeaterMetalBassDrumUp
`'bass-double`		U+E7A0	pictBeaterDoubleBassDrumUp
`'stick`		U+E7E8	pictDrumStick
`'stick-snare`		U+E7D1	pictBeaterSnareSticksUp
`'stick-jazz`		U+E7D3	pictBeaterJazzSticksUp
`'hammer-wood`		U+E7CB	pictBeaterHammerWoodUp
`'hammer-plastic`		U+E7CD	pictBeaterHammerPlasticUp
`'hammer-metal`		U+E7CF	pictBeaterHammerMetalUp

'wound-hard		U+E7B3	pictWoundHardUp
'wound-soft		U+E7B7	pictWoundSoftUp
'metal		U+E7C7	pictBeaterMetalUp
'brass-mallets		U+E7D9	pictBeaterBrassMalletsUp
'triangle		U+E7D5	pictBeaterTriangleUp
'triangle-plain		U+E7EF	pictBeaterTrianglePlain
'wire-brushes		U+E7D7	pictBeaterWireBrushesUp
'superball		U+E7AE	pictBeaterSuperballUp
'mallet		U+E7DF	pictBeaterMallet
'metal-hammer		U+E7E0	pictBeaterMetalHammer
'hammer		U+E7E1	pictBeaterHammer
'hand		U+E7E3	pictBeaterHand
'finger		U+E7E4	pictBeaterFinger
'fist		U+E7E5	pictBeaterFist
'fingernails		U+E7E6	pictBeaterFingernails

```
\ekm-fader LEVEL ORIENTATION
\ekm-midi LEVEL ORIENTATION
```

Draw a fader (volume control) and a MIDI controller, respectively, as markup.
LEVEL is drawn as a label next to the control according to ORIENTATION . `#f` draws no label.

- ≥ 0 is a percent value.
- < 0 is a decibel (dB) value, e.g. -6.0 is equivalent to 50.

For the thumb position, LEVEL is rounded to the nearest integral percent value, limited to 100. If a symbol is defined exactly for this value, this symbol is drawn. Else if an empty control and a thumb symbol are defined, they are combined. Else the symbol for the value nearest to LEVEL is drawn.
Used properties:

- `label-format` (#f) : `#f` uses "~a%" for percent and "~adB" for decibel values.
- `font-size` (0)
- `label-size` (-4) relative to the font size.
- `padding` (0.3)

`\ekm-fader`

0		U+EB2E	elecVolumeLevel0
	:		
100		U+EB33	elecVolumeLevel100
		U+EB2C	elecVolumeFader
		U+EB2D	elecVolumeFaderThumb

`\ekm-midi`

0		U+EB36	elecMIDIController0
	:		
100		U+EB3B	elecMIDIController100

with Ekmelos

`\ekm-midi`		U+F6D2	elecMIDIController
		U+F6D3	elecMIDIControllerThumb

Examples

ORIENTATION	N	NE	E	SE	S	SW	W	NW
`\ekm-fader`								
								
`\ekm-midi`								
								

`\ekm-arrow STYLE ORIENTATION`

Draw an arrow or arrow head of STYLE according to ORIENTATION as markup.

Arrows can have glyphs for several orientations. The remaining orientations are achieved by flipping or rotating through 90° or 45°. All following six styles have glyphs for the 8 orientations `N` … `NW`.

STYLE

`'black`	↑	U+EB60	arrowBlackUp
`'white`	⇧	U+EB68	arrowWhiteUp
`'open`	↑	U+EB70	arrowOpenUp
`'black-head`	▲	U+EB78	arrowheadBlackUp
`'white-head`	△	U+EB80	arrowheadWhiteUp
`'open-head`	∧	U+EB88	arrowheadOpenUp

`\ekm-arrow-head AXIS DIRECTION FILLED`

Draw an arrow head as markup, i.e. `black-head` if FILLED is a true value, else `open-head` .

with Ekmelos

`'simple`	↑	U+2191
`'double`	⇑	U+21D1
`'triple`	⟰	U+290A
`'quadruple`	⟰	U+27F0
`'black-wide`	⬆	U+2B06
`'white-wide`	⇧	U+21E7
`'triangle`	↑	U+2B61
`'triangle-bar`	↑	U+2B71
`'two-headed`	↑	U+2BED
`'dashed`	⇡	U+21E1
`'triangle-dashed`	⇡	U+2B6B
`'opposite`	⇅	U+21C5
`'triangle-opposite`	⇅	U+2B81
`'paired`	⇈	U+21C8
`'triangle-paired`	⇈	U+2B85
`'bent-tip`	↱	U+21B1
`'long-bent-tip`	⮡	U+2BA3
`'curving`	⤴	U+2934
`'equilateral-head`	⮝	U+2B9D
`'three-d-head`	⮜	U+2B99
`'black-triangle`	▲	U+25B2
`'white-triangle`	△	U+25B3
`'black-small-triangle`	▴	U+25B4
`'white-small-triangle`	▵	U+25B5
`'half-circle`	◗	U+2BCA
`'circle-half-black`	◓	U+25D3
`'square-half-black`	◧	U+2B12
`'diamond-half-black`	◈	U+2B18
`'circle-quarters`	◔	U+25D4